

PANDAS 개발자를 위한

PySpark 핵심 가이드

분산 DataFrame의 실행 모델부터
성능 튜닝, 메서드 치트 시트까지



Spark 4.x 기준 · 모든 예제 실행 검증 (PySpark 4.2.0)
모바일 판형 · 2026-09

차례

들어가며

1 Spark는 무엇이고 어떻게 생겼나

- 1.1 pandas와 결정적으로 다른 세 가지
- 1.2 구성 요소
- 1.3 Python은 어디서 도는가
- 1.4 SparkSession: 모든 것의 입구
- 1.5 Spark Connect
- 1.6 핵심 정리

2 실행 모델: 게으름, 계획, 셔플

- 2.1 변환(Transformation)과 액션(Action)
- 2.2 계획 들여다보기: `explain()`
- 2.3 파티션: 병렬성의 단위
- 2.4 좁은 변환과 넓은 변환
- 2.5 Job, Stage, Task
- 2.6 핵심 정리

3 DataFrame 기초

- 3.1 DataFrame 만들기
- 3.2 주요 데이터 타입
- 3.3 들여다보기
- 3.4 Column 표현식
- 3.5 행과 열 다루기
- 3.6 타입 변환과 ANSI 모드
- 3.7 핵심 정리

4 데이터 읽고 쓰기

- 4.1 기본 형태

- 4.2 예제 데이터 준비
- 4.3 CSV: 가장 흔하고 가장 위험한
- 4.4 JSON
- 4.5 Parquet: 기본값으로 삼을 포맷
- 4.6 쓰기 모드
- 4.7 테이블로 저장하기
- 4.8 테이블 포맷: Delta Lake, Iceberg
- 4.9 JDBC: 데이터베이스 읽기
- 4.10 핵심 정리

5 내장 함수로 변환하기

- 5.1 조건식: `when` / `otherwise`
- 5.2 `null` 다루기
- 5.3 문자열
- 5.4 날짜와 시간
- 5.5 배열, 맵, 구조체
- 5.6 JSON 문자열 다루기
- 5.7 핵심 정리

6 집계와 조인

- 6.1 예제 데이터
- 6.2 `groupBy` + `agg`
- 6.3 조인
- 6.4 조인 전략: Spark는 어떻게 조인하는가
- 6.5 `union`
- 6.6 핵심 정리

7 원도 함수

- 7.1 원도 정의
- 7.2 순위
- 7.3 이전·다음 행: `lag` / `lead`
- 7.4 누적과 이동 집계: 프레임

- 7.5 원도와 성능
- 7.6 시간 버킷 집계: F.window
- 7.7 핵심 정리

8 Spark SQL

- 8.1 임시 뷰
- 8.2 DataFrame을 SQL에 직접 넘기기
- 8.3 파라미터 바인딩
- 8.4 섞어 쓰기
- 8.5 CTE와 서브쿼리
- 8.6 SQL 파이프 문법 (4.0+)
- 8.7 카탈로그와 테이블
- 8.8 핵심 정리

9 UDF와 pandas 연동

- 9.1 Python UDF: 왜 느린가
- 9.2 pandas UDF: 벡터화된 UDF
- 9.3 얼마나 차이 나는가
- 9.4 DataFrame 단위의 pandas 함수
- 9.5 Spark ↔ pandas 변환
- 9.6 pandas API on Spark
- 9.7 그 밖의 Python 확장 지점
- 9.8 핵심 정리

10 성능 튜닝

- 10.1 캐싱
- 10.2 파티션 수 조절
- 10.3 AQE: 실행 중 재최적화
- 10.4 데이터 스큐
- 10.5 셔플 줄이기 체크리스트
- 10.6 작은 파일 문제
- 10.7 메모리와 스펠

- 10.8 자주 만지는 설정
- 10.9 안티패턴 모음
- 10.10 핵심 정리

11 RDD: 알아야 할 저수준 API

- 11.1 RDD라는 이름의 뜻
- 11.2 맛보기
- 11.3 기억해 둘 RDD 개념
- 11.4 브로드캐스트 변수와 어큐뮬레이터
- 11.5 DataFrame ↔ RDD
- 11.6 핵심 정리

12 Structured Streaming

- 12.1 핵심 아이디어: 끝없이 늘어나는 테이블
- 12.2 예제: 폴더에 쌓이는 JSON 이벤트
- 12.3 출력 모드
- 12.4 이벤트 시간과 워터마크
- 12.5 트리거
- 12.6 체크포인트와 정확히 한 번
- 12.7 Kafka와 foreachBatch
- 12.8 쿼리 관리
- 12.9 핵심 정리

13 배포, 테스트, 디버깅

- 13.1 애플리케이션 구조
- 13.2 spark-submit
- 13.3 테스트
- 13.4 Spark UI 읽는 법
- 13.5 에러 메시지 읽기
- 13.6 자주 만나는 문제
- 13.7 핵심 정리

부록 A. pandas → PySpark 번역표

만들기·보기

선택·변환

필터·정렬·중복

결측값

집계

원도(transform·shift·rank·rolling)

결합

그 밖의 차이

부록 B. 메서드 치트 시트

B.1 세션·설정

B.2 읽기

B.3 쓰기

B.4 살펴보기

B.5 컬럼 선택·추가·변경

B.6 행 필터·정렬·중복·샘플

B.7 Column 메서드

B.8 조건·null

B.9 수학·통계

B.10 문자열

B.11 날짜·시간

B.12 배열·맵·구조체·JSON

B.13 집계

B.14 원도

B.15 조인·집합 연산

B.16 파티션·캐시·실행

B.17 SQL·카탈로그

B.18 UDF·pandas

B.19 스트리밍

B.20 테스트·기타

들어가며

이 책은 **pandas에 익숙한 Python 개발자**가 PySpark를 빠르게, 그러나 제대로 익히도록 쓰였습니다. Python 문법이나 DataFrame이라는 개념 자체는 설명하지 않습니다. 대신 “pandas처럼 생겼는데 왜 다르게 동작하는가”에 지면을 씁니다.

대상 버전. Spark 4.x(예제는 4.2.0에서 실제로 실행해 출력을 붙였습니다). 3.5에서도 대부분 동작하며, 4.0에서 바뀐 부분은 따로 표시합니다.

읽는 순서.

- 1-2장: 아키텍처와 실행 모델. **가장 중요합니다.** 여기를 건너뛰면 나머지가 전부 “마법”으로 보입니다.
- 3-8장: DataFrame API와 SQL. 매일 쓰는 도구입니다.
- 9-10장: pandas 연동과 성능. 실무에서 차이가 나는 곳입니다.
- 11-13장: RDD, 스트리밍, 배포·운영.
- 부록: pandas→PySpark 번역표, 메서드 치트 시트.

표기. 모든 예제는 다음 import를 전제합니다.

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql import types as T
from pyspark.sql.window import Window
```

F, T 별칭은 PySpark 커뮤니티의 사실상 표준입니다. from pyspark.sql.functions import * 는 sum, max, round 같은

Python 내장 함수를 덮어쓰므로 쓰지 마십시오.

TIP

로컬 환경 준비. Java 17 또는 21이 필요합니다. `pip install "pyspark[sql]"` 한 줄이면 클러스터 없이 노트북에서 모든 예제를 돌릴 수 있습니다. `[sql]` 은 `pandas-pyarrow`를 함께 설치합니다. PySpark 4.2는 아직 `pandas 3.x`를 완전히 지원하지 않아 경고를 띄우므로, `pandas`는 `2.x` 버전을 권합니다.

Spark는 무엇이고 어떻게 생겼나

1.1 pandas와 결정적으로 다른 세 가지

pandas DataFrame은 한 프로세스의 메모리에 있는 실제 데이터입니다. `df["a"] + 1` 을 쓰는 순간 계산이 끝나고 결과가 메모리에 생깁니다.

Spark DataFrame은 셋 다 다릅니다.

1. **분산되어 있습니다.** 데이터는 여러 머신의 여러 프로세스에 *파티션* 단위로 쪼개져 있습니다. 한 곳에 모이는 일은 원칙적으로 없습니다.
2. **게으릅니다(lazy).** `df.filter(...)` 는 아무것도 계산하지 않고 “이렇게 계산하겠다”는 **계획**만 반환합니다. 실제 계산은 `count()`, `show()`, `write` 같은 **액션**을 부를 때 일어납니다.
3. **불변입니다.** 모든 변환은 새 DataFrame(=새 계획)을 반환합니다. `inplace=True` 같은 것은 없습니다.

한마디로, **Spark DataFrame은 데이터가 아니라 쿼리입니다.** SQL의 `VIEW` 에 더 가깝습니다. 이 관점만 잡으면 이 책의 절반은 이해한 셈입니다.

1.2 구성 요소

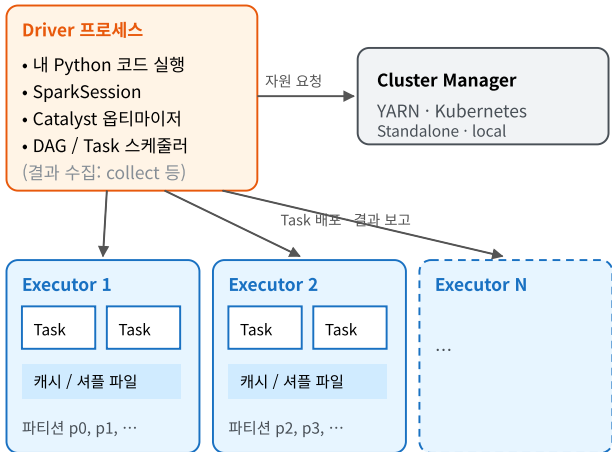


그림 1-1. Spark 애플리케이션의 구성

Driver. 여러분의 Python 스크립트가 실행되는 프로세스입니다. SparkSession을 갖고 있으며, DataFrame 연산을 받아 **논리 계획** → **최적화** → **물리 계획**으로 바꾸고, 이를 작은 작업(Task)으로 쪼개 Executor에 나눠 줍니다. Driver는 데이터를 거의 만지지 않습니다. 단, `collect()` · `toPandas()` 를 부르면 결과 전체가 Driver 메모리로 옵니다. Driver OOM의 대부분이 여기서 납니다.

Executor. 워커 노드에서 도는 JVM 프로세스입니다. 파티션 하나를 처리하는 단위가 **Task**이고, Executor는 코어 수만큼 Task를 동시에 돌립니다. 캐시된 데이터와 셔플 중간 파일도 Executor가 들고 있습니다.

Cluster Manager. Executor를 띄울 자원(CPU·메모리)을 빌려주는 쪽입니다. YARN, Kubernetes, Spark Standalone이 있고, 학습용 `local[*]` 모드는 Driver 프로세스 안의 스레드를 Executor처럼 씁니다.

NOTE

분산 DB 경험과의 대응. Driver는 쿼리 코디네이터, Executor는 데이터 노드의 쿼리 워커와 비슷합니다. 다른 점은, Spark는 **저장소를 소유하지 않는다**는 것입니다. 데이터는 S3·HDFS·DB 등 바깥에 있고, Spark는 매 작업마다 읽어 와서 계산만 합니다. “스토리지와 컴퓨트의 분리”가 기본값입니다.

1.3 Python은 어디서 도는가

Spark 엔진은 JVM(Scala)으로 짜여 있습니다. PySpark의 DataFrame API는 **JVM 객체를 조종하는 얇은 리모컨**입니다. `df.filter(F.col("x") > 1)` 은 Python에서 식을 만들 뿐이고, 실제 필터링은 JVM의 최적화된 코드가 수행합니다. 그래서 DataFrame API만 쓰면 PySpark도 Scala만큼 빠릅니다.

예외가 **Python UDF**입니다. 사용자 정의 Python 함수는 JVM이 실행할 수 없으므로, Executor 옆에 Python 워커 프로세스를 띄워 행을 직렬화해 보내고, 결과를 다시 받아옵니다. 이 왕복이 비쌉니다. 9장에서 다룹니다.

1.4 SparkSession: 모든 것의 입구

```
from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
        .appName("my-app")
        .master("local[4]") # 로컬 스레드 4개
        .config("spark.sql.shuffle.partitions", "8")
        .getOrCreate()
)
print(spark.version)
```

출력
4.2.0

- `getOrCreate()` 는 이미 세션이 있으면 그것을 돌려줍니다. 노트북에서 설정을 바꿔 다시 실행해도 **기존 세션이 재사용되어 설정이 안 먹는** 흔한 함정이 여기서 생깁니다. 이미 떠 있는 세션의 정적 설정을 바꾸려면 `spark.stop()` 후 다시 만드십시오.
- `.master(...)` 는 로컬 학습 때만 씁니다. 운영에서는 `spark-submit` 이나 플랫폼(Databricks, EMR 등)이 정해 주므로 코드에 박지 않습니다.
- `spark.sparkContext` 는 RDD 시절의 저수준 진입점입니다. 요즘은 로그 레벨 설정 정도에만 씁니다.

1.5 Spark Connect

Spark 3.4부터 도입되고 4.x에서 본격화된 **클라이언트-서버 모드**입니다. Driver가 원격 서버에 있고, 내 Python

프로세스는 gRPC로 **미해결 논리 계획(unresolved plan)** 만 보냅니다.

```
spark = (SparkSession.builder
        .remote("sc://spark-host:15002")
        .getOrCreate())
```

- 클라이언트가 가볍습니다(`pip install pyspark-client`, JVM 불필요).
- 클라이언트가 죽어도 서버는 멀쩡합니다. 여러 사용자가 한 서버를 공유하기 좋습니다.
- **제약:** `sparkContext` 와 RDD API를 쓸 수 없습니다. DataFrame/SQL 중심으로 코드를 쓰는 습관이 곧 이식성입니다.

Databricks의 서버리스·공유 클러스터도 내부적으로 이 모드입니다. 이 책의 예제는 11장(RDD)을 빼면 Connect에서도 그대로 돌아갑니다.

1.6 핵심 정리

- Spark DataFrame은 **분산·계으름·불변**. 데이터가 아니라 쿼리입니다.
- Driver가 계획하고 Executor가 실행합니다. `collect()` 는 Driver로 데이터를 끌고 옵니다.
- DataFrame API는 JVM에서 실행되어 빠르고, Python UDF만 예외적으로 느립니다.

CHAPTER 2

실행 모델: 게으름, 계획, 셔플

이 장은 이론처럼 보이지만 **성능 문제의 90%가 여기서 설명됩니다.** 천천히 읽으십시오.

2.1 변환(Transformation)과 액션(Action)

DataFrame 메서드는 두 부류입니다.

구분	반환	예	실행
변환	새 DataFrame	<code>select , filter , withColumn , join , groupBy().agg() , orderBy</code>	안 함 (계획만)
액션	값·부수효과	<code>count , show , collect , take , first , toPandas , write...save()</code>	즉시 실행

```
from pyspark.sql import functions as F

df = spark.range(0, 10_000_000)          # 0..9,999,999
big = df.withColumn("sq", F.col("id") * F.col("id"))
evens = big.filter(F.col("id") % 2 == 0)
# 여기까지 실행된 계산: 없음
print(evens.count())                    # 이 순간 전부 실행
```

```
출력
5000000
```

`evens` 를 만드는 데는 몇 밀리초도 걸리지 않습니다. 천만 행을 제공하고 거르는 일은 `count()` 를 부를 때 한꺼번에 일어납니다.

왜 게으른가

게으름은 **최적화 기회**입니다. 액션 시점에는 Spark가 전체 파이프라인을 한눈에 볼 수 있으므로,

- 필터를 가능한 한 앞으로 당기고(predicate pushdown), 파일 읽을 때부터 걸러냅니다.
- 쓰지 않는 컬럼은 아예 읽지 않습니다(column pruning).
- 연속된 `withColumn` · `filter` 를 하나의 루프로 합칩니다 (whole-stage codegen).

pandas에서는 한 줄 한 줄이 중간 결과를 메모리에 만들지만, Spark에서는 중간 결과가 **없습니다**.

반복 액션의 함정

반대로 말하면, 액션을 부를 때마다 계획 전체가 처음부터 다시

실행됩니다.

```
cleaned = raw.filter(...).join(...).groupBy(...).agg(...)
cleaned.count()           # 전체 실행 1회
cleaned.show()           # 전체 실행 2회 (처음부터!)
cleaned.write.parquet()   # 전체 실행 3회
```

같은 결과를 여러 번 쓸 거라면 `cache()` 로 한 번 계산한 결과를 붙잡아 두어야 합니다(10장). 디버깅한다고 중간중간 `count()` 를 넣는 습관이 파이프라인을 몇 배 느리게 만드는 주범입니다.

2.2 계획 들여다보기: `explain()`

```
df1 = spark.range(100).withColumn("g", F.col("id") % 3)
q = (df1.filter(F.col("id") > 10)
      .groupBy("g").agg(F.sum("id").alias("s")))
q.explain()
```

출력

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[g#9L], functions=[sum(id#8L)])
   +- Exchange hashpartitioning(g#9L, 8), ENSURE_REQUIREMENTS, [plan_id=58]
      +- HashAggregate(keys=[g#9L], functions=[partial_sum(id#8L)])
         +- Project [id#8L, (id#8L % 3) AS g#9L]
            +- Filter (id#8L > 10)
               +- Range (0, 100, step=1, splits=4)
```

아래에서 위로 읽습니다. `Range` → `Filter` → `Project` → `HashAggregate(partial)` → **`Exchange`** → `HashAggregate(final)`.

`AdaptiveSparkPlan isFinalPlan=false` 는 실행 중에 계획이 바뀔 수 있다는 뜻입니다(AQE, 10장). 옵티마이저의 각 단계를 모두 보려면 `q.explain(mode="extended")`, 읽기 좋은 형태는 `mode="formatted"` 입니다.

Catalyst 옵티마이저

여러분이 쓴 DataFrame 코드는 다음 단계를 거칩니다.

1. **미해결 논리 계획**: 컬럼 이름이 실제 있는지 아직 모름.
2. **분석된 논리 계획**: 카탈로그/스키마와 대조해 이름·타입 확정. 오타가 있으면 여기서 `AnalysisException`.
3. **최적화된 논리 계획**: 규칙 기반 최적화(필터 푸시다운, 상수 접기, 불필요한 프로젝션 제거 등).
4. **물리 계획**: 조인 알고리즘 선택 등. 통계를 참고합니다 (CBO).
5. **코드 생성**: JVM 바이트코드.

SQL로 쓰든 DataFrame API로 쓰든 2단계부터 똑같습니다.

SQL과 DataFrame API 사이에 성능 차이는 없습니다. 편한 쪽을 쓰면 됩니다.

NOTE

분석(2단계)은 변환을 만드는 시점에 **즉시** 일어납니다. 그래서 없는 컬럼을 참조하면 액션 전에도 바로 에러가 납니다.
“게으르다”는 것은 *데이터 처리*가 게으르다는 뜻입니다.

2.3 파티션: 병렬성의 단위

DataFrame은 **파티션**들의 모음입니다. 파티션 하나 = Task

하나 = 코어 하나가 처리하는 덩어리.

```
df2 = spark.range(0, 1000, numPartitions=6)
print(df2.rdd.getNumPartitions())
```

출력
6

- 파일을 읽으면 대략 **128MB**(`spark.sql.files.maxPartitionBytes`)당 파티션 하나가 생깁니다.
- 파티션이 너무 적으면 코어가 놀고, 한 Task가 메모리를 넘칩니다.
- 너무 많으면 Task 스케줄링 오버헤드와 작은 파일 문제가 생깁니다.
- 경험칙: **파티션당 100-200MB**, 파티션 수는 **총 코어 수의 2-4배**.

2.4 좁은 변환과 넓은 변환

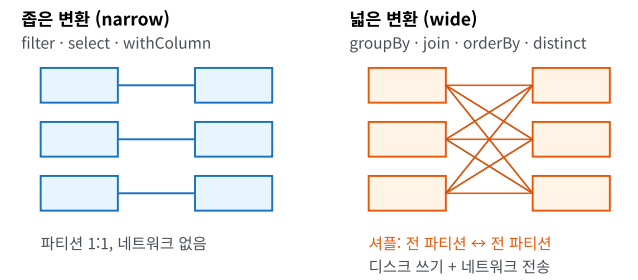


그림 2-1. 좁은 변환과 넓은 변환

좁은 변환은 입력 파티션 하나가 출력 파티션 하나만 만듭니다. 각 Executor가 자기 데이터만 보고 끝냅니다. 여러 개가

이어져도 한 번에 파이프라이닝됩니다.

넓은 변환은 같은 키를 가진 행을 한곳에 모아야 합니다. `group By("user")` 라면 모든 파티션에 흩어진 `user=42` 의 행이 한 파티션으로 가야 하지요. 이 재배치가 **셔플(shuffle)** 이고, `explain()` 의 `Exchange` 가 바로 셔플입니다.

셔플은 Spark에서 가장 비싼 연산입니다.

1. 각 Executor가 행을 키 해시로 나눠 **로컬 디스크에 씁니다** (shuffle write).
2. 다음 단계의 Task들이 **네트워크로 모든 Executor에서** 자기 몫을 가져옵니다(shuffle read).
3. 직렬화·압축·정렬 비용이 따릅니다.

셔플 후 파티션 수는 `spark.sql.shuffle.partitions` (기본 200) 가 정합니다. 요즘은 AQE가 실행 중에 작은 파티션을 합쳐 주므로 이 값에 덜 민감합니다.

2.5 Job, Stage, Task

액션 하나가 **Job** 하나를 만듭니다(가끔 여러 개). Job은 셔플 경계에서 **Stage**로 나뉩니다. Stage 안은 좁은 변환만 있으니 파이프라이닝되고, Stage는 파티션 수만큼의 **Task**로 실행됩니다.

```
q.collect()                                ← Job 1개
├─ Stage 0: Range → Filter → partial agg
│   · Task 수 = 입력 파티션 수
│   · 끝에서 shuffle write
└─ Stage 1: shuffle read → final agg
    · Task 수 = 셔플 파티션 수
```

Spark UI(로컬은 `http://localhost:4040`)의 Jobs/Stages 탭이 정확히 이 구조를 보여 줍니다. 느린 작업을 조사할 때는 13장의 요령대로 **가장 오래 걸린 Stage** → **그 안에서 가장 오래 걸린 Task**를 찾아 내려가십시오.

2.6 핵심 정리

- 변환은 계획, 액션은 실행. 액션마다 처음부터 다시 계산합니다.
- `explain()` 으로 계획을 읽는 습관을 들이십시오. `Exchange` = 셔플.
- 파티션 = 병렬성 단위. 셔플은 Stage를 가르는 비싼 경계입니다.
- 성능 튜닝은 대부분 **셔플을 줄이거나, 셔플을 고르게 만드는 일**입니다.

DataFrame 기초

3.1 DataFrame 만들기

실무에서는 대부분 파일·테이블을 읽지만(4장), 학습과

테스트에서는 직접 만드는 일이 많습니다.

```
from pyspark.sql import functions as F
from pyspark.sql import types as T

rows = [
    (1, "kim", "Seoul", 34, 5200.0),
    (2, "lee", "Busan", 28, 4100.0),
    (3, "park", "Seoul", 45, None),
    (4, "choi", "Daegu", 39, 6100.0),
    (5, "jung", "Busan", 23, 3000.0),
]

schema = ("id INT, name STRING, city STRING, "
          "age INT, salary DOUBLE")
people = spark.createDataFrame(rows, schema)
people.show()
people.printSchema()
```

출력

```
+---+-----+-----+-----+
| id|name| city|age|salary|
+---+-----+-----+-----+
|  1| kim|Seoul| 34|5200.0|
|  2| lee|Busan| 28|4100.0|
|  3|park|Seoul| 45|  NULL|
|  4|choi|Daegu| 39|6100.0|
|  5|jung|Busan| 23|3000.0|
+---+-----+-----+-----+
```

```
root
|-- id: integer (nullable = true)
|-- name: string (nullable = true)
|-- city: string (nullable = true)
|-- age: integer (nullable = true)
|-- salary: double (nullable = true)
```

스키마는 **DDL 문자열**("id INT, name STRING")로 주는 것이 가장 간결합니다. 생략하면 Spark가 데이터를 훑어 타입을 추론하는데, 파이썬 int 는 LONG (bigint)이 되고 None 만 있는

컬럼은 추론에 실패합니다. 테스트 코드라도 스키마를 명시하는 습관이 좋습니다.

`StructType` 으로 쓰면 장황하지만 프로그램으로 조립하기 좋습니다.

```
schema2 = T.StructType([
    T.StructField("id", T.IntegerType(), nullable=False),
    T.StructField("tags", T.ArrayType(T.StringType())),
    T.StructField("attrs", T.MapType(
        T.StringType(), T.StringType())),
])
spark.createDataFrame(
    [(1, ["a", "b"], {"k": "v"})], schema2
).printSchema()
```

출력

```
root
|-- id: integer (nullable = false)
|-- tags: array (nullable = true)
|   |-- element: string (containsNull = true)
|-- attrs: map (nullable = true)
|   |-- key: string
|   |-- value: string (valueContainsNull = true)
```

pandas DataFrame도 바로 넘길 수 있습니다. Arrow가 켜져 있으면(`spark.sql.execution.arrow.pyspark.enabled` , 4.x 기본 활성화) 빠르게 변환됩니다.

```
import pandas as pd
pdf = pd.DataFrame({"x": [1, 2, 3],
                    "y": ["a", "b", "c"]})
sdf = spark.createDataFrame(pdf)
sdf.show()
```

출력

```
+---+---+
| x| y|
+---+---+
| 1| a|
| 2| b|
| 3| c|
+---+---+
```

3.2 주요 데이터 타입

Spark SQL	PySpark 클래스	Python 값
INT / BIGINT	IntegerType / LongType	int
DOUBLE / FLOAT	DoubleType / FloatType	float
DECIMAL(p,s)	DecimalType(p, s)	decimal.Decimal
STRING	StringType	str
BOOLEAN	BooleanType	bool
DATE	DateType	datetime.date
TIMESTAMP	TimestampType	datetime.datetime
TIMESTAMP_NTZ	TimestampNTZType	datetime (타임존 없음)
ARRAY<T>	ArrayType(T)	list

Spark SQL	PySpark 클래스	Python 값
MAP<K,V>	MapType(K, V)	dict
STRUCT<...>	StructType([...])	Row
VARIANT (4.0+)	VariantType	반정형 JSON

TIMESTAMP 는 내부적으로 UTC 순간값을 저장하고 표시할 때 `spark.sql.session.timeZone` 으로 바꿉니다. 이 책의 예제는 `Asia/Seoul` 로 설정했습니다. 운영 환경에서는 이 설정을 명시적으로 고정하십시오. 클러스터 기본 타임존에 결과가 휘둘리는 사고가 잦습니다.

3.3 들여다보기

```
print(people.columns)
print(people.dtypes)
people.show(2, truncate=False)
people.show(1, vertical=True)
```

출력

```
['id', 'name', 'city', 'age', 'salary']
[('id', 'int'), ('name', 'string'), ('city', 'string'), ('age', 'int'), ('salary', 'double')]
+---+---+---+---+
|id|name|city|age|salary|
+---+---+---+---+
|1|kim|Seoul|34|5200.0|
|2|lee|Busan|28|4100.0|
+---+---+---+---+
only showing top 2 rows
-RECORD 0-----
id      | 1
name    | kim
city    | Seoul
age     | 34
salary  | 5200.0
only showing top 1 row
```

- `show(n)` : 상위 `n`행을 **Driver에서 출력**합니다. 액션입니다.
- `take(n)` / `head(n)` : `Row` 객체 리스트를 반환합니다.
- `limit(n)` : `n`행짜리 **DataFrame**을 반환합니다(변환).
- `collect()` : 전체 행을 Driver로 가져옵니다. 큰 데이터에서는 절대 금지입니다.
- `count()` : 행 수. 전체 스캔이 일어납니다.

```
r = people.first()
print(r)
print(r.name, r["age"], r.asDict())
```

출력

```
Row(id=1, name='kim', city='Seoul', age=34, salary=5200.0)
kim 34 {'id': 1, 'name': 'kim', 'city': 'Seoul', 'age': 34, 'salary': 5200.0}
```

Row 는 namedtuple처럼 속성과 키 둘 다로 접근합니다.

3.4 Column 표현식

Spark 연산의 기본 단위는 값이 아니라 **Column 표현식**입니다. 컬럼을 가리키는 방법은 넷입니다.

people.select("age")	# 문자열
people.select(people.age)	# 속성
people.select(people["age"])	# 인덱싱
people.select(F.col("age"))	# 함수 (가장 권장)

F.col("age") 는 특정 DataFrame에 묶이지 않은 표현식이라 재사용과 체이닝에 유리합니다. people.age 는 조인 후 이름이 겹칠 때 어느 쪽 컬럼인지 지정하는 용도로 씁니다. 속성 방식은 count , name 같은 DataFrame 메서드 이름과 충돌하므로 피하십시오.

표현식은 연산자로 조립합니다.

```
people.select(
    "name",
    F.round(F.col("salary") * 1.1, 1).alias("raised"),
    (F.col("age") >= 30).alias("senior"),
    F.lit("KR").alias("country"),
    F.concat_ws("-", "city", "name").alias("key"),
).show()
```

출력

name	raised	senior	country	key
kim	5720.0	true	KR	Seoul-kim
lee	4510.0	false	KR	Busan-lee
park	NULL	true	KR	Seoul-park
choi	6710.0	true	KR	Daegu-choi
jung	3300.0	false	KR	Busan-jung

주의

논리 연산자는 &, |, ~ 이고 괄호가 필수입니다. Python의 and / or / not 은 Column에 쓸 수 없습니다(에러). 또 & 가 > = 보다 우선순위가 높기 때문에 `F.col("a") > 1 & F.col("b") < 2` 는 엉뚱하게 해석됩니다. 조건 하나하나를 반드시 괄호로 감싸십시오. pandas의 불리언 인덱싱과 같은 규칙입니다.

3.5 행과 열 다루기

```
result = (
    people
    .filter((F.col("age") >= 25)
        & F.col("salary").isNotNull())
    .withColumn("band", F.floor(F.col("age") / 10) * 10)
    .withColumnRenamed("salary", "pay")
    .drop("id")
    .orderBy(F.col("pay").desc())
)
result.show()
```

출력

name	city	age	pay	band
choi	Daegu	39	6100.0	30
kim	Seoul	34	5200.0	30
lee	Busan	28	4100.0	20

- filter 와 where 는 같습니다. SQL 문자열도 받습니다: where("age >= 25") .
- withColumn(name, expr) : 컬럼 추가·교체. 같은 이름이면 덮어씁니다.
- orderBy / sort : 전역 정렬 = **셔플**. 큰 데이터에서 “그냥 보기 좋게” 정렬하지 마십시오.

여러 컬럼을 한 번에

withColumn 을 반복문에서 수십 번 부르면 계획이 층층이 쌓여 분석 시간이 급격히 늘어납니다(심하면 StackOverflowError).

여러 컬럼은 `withColumns (3.3+)` 또는 `select` 한 번으로 만드십시오.

```
exprs = {f"age_plus_{i}": F.col("age") + i
         for i in range(3)}
people.withColumns(exprs).select(
    "name", *exprs.keys()).show(2)
```

출력

```
+---+-----+-----+-----+
|name|age_plus_0|age_plus_1|age_plus_2|
+---+-----+-----+-----+
| kim|      34|      35|      36|
| lee|      28|      29|      30|
+---+-----+-----+-----+
only showing top 2 rows
```

```
# 모든 문자열 컬럼 대문자화: select + 리스트 컴프리헨션
people.select([
    F.upper(c).alias(c) if t == "string" else F.col(c)
    for c, t in people.dtypes
]).show(2)
```

출력

```
+---+-----+-----+-----+
| id|name| city|age|salary|
+---+-----+-----+-----+
|  1| KIM|SEOUL| 34|5200.0|
|  2| LEE|BUSAN| 28|4100.0|
+---+-----+-----+-----+
only showing top 2 rows
```

이름 변경도 한 번에: `withColumnsRenamed({"a": "x", "b": "y"}) (3.4+)`.

SQL 식 문자열

간단한 식은 SQL 문자열이 더 읽기 좋을 때가 있습니다.

```
people.selectExpr(  
    "name",  
    "salary * 12 AS annual",  
    "CASE WHEN age < 30 THEN 'young' "  
    "ELSE 'adult' END AS grp",  
)  
.show(3)  
people.filter("city IN ('Seoul', 'Busan')").count()
```

출력

```
+---+-----+---+  
|name| annual| grp|  
+---+-----+---+  
| kim|62400.0|adult|  
| lee|49200.0|young|  
|park|  NULL|adult|  
+---+-----+---+  
only showing top 3 rows
```

중복 제거

```
people.select("city").distinct().show()
people.dropDuplicates(["city"]) \
    .select("city", "name").show()
```

출력

```
+-----+
| city|
+-----+
|Seoul|
|Busan|
|Daegu|
+-----+
```

```
+-----+-----+
| city|name|
+-----+-----+
|Busan| Lee|
|Daegu|choi|
|Seoul| kim|
+-----+-----+
```

`dropDuplicates(subset)` 은 키별로 **아무 행 하나**를 남깁니다.
어느 행이 남을지는 보장되지 않습니다. “가장 최근 행”처럼
기준이 있다면 7장의 윈도 함수(`row_number`)를 쓰십시오.

3.6 타입 변환과 ANSI 모드

```
raw = spark.createDataFrame(
    [("1",), ("2.5",), ("abc",)], "v STRING")
raw.select(
    "v",
    F.col("v").try_cast("double").alias("d"),
    F.col("v").try_cast("int").alias("i"),
).show()
```

출력

```
+---+---+---+
| v | d | i |
+---+---+---+
| 1 | 1.0 | 1 |
| 2.5 | 2.5 | NULL |
| abc | NULL | NULL |
+---+---+---+
```

NOTE

Spark 4.0의 큰 변화: ANSI 모드 기본 활성화. 3.x에서는 "abc" 를 int 로 cast 하면 조용히 null 이 되었고, 정수 오버플로도 조용히 넘어갔습니다. 4.0부터는 `spark.sql.ansi.enabled=true` 가 기본이라 **예외가 납니다.** 실패를 null로 받고 싶으면 `try_cast`, `try_divide`, `try_add`, `try_to_timestamp` 같은 `try_*` 계열을 쓰십시오. 3.x 코드를 4.x로 올릴 때 가장 흔히 터지는 부분입니다.

```
raw.select(F.col("v").cast("int")).collect()
```

출력

```
NumberFormatException: [CAST_INVALID_INPUT] The value 'abc' of the
type "STRING" cannot be cast to "INT" because it is malformed. Cor
rect the value as per the syntax, or change its ta...
```

3.7 핵심 정리

- 스키마는 DDL 문자열로 명시하십시오.
- 연산은 `F.col(...)` 기반 Column 표현식으로 조립합니다.
- 조건은 `&·|·~` 와 괄호. `and / or` 는 안 됩니다.
- 여러 컬럼은 `withColumns / select` 한 번에.
- 4.x는 ANSI 모드. 관대한 변환은 `try_*` 로.

데이터 읽고 쓰기

4.1 기본 형태

읽기는 `spark.read` (`DataFrameReader`), 쓰기는 `df.write` (`DataFrameWriter`)에서 시작합니다. 모양은 늘 같습니다.

```
df = (spark.read
      .format("csv")      # json, parquet, jdbc, delta...
      .option("header", True)
      .schema(schema)
      .load("s3://bucket/path/"))

(df.write
 .format("parquet")
 .mode("overwrite")
 .partitionBy("dt")
 .save("s3://bucket/out/"))
```

`spark.read.csv(path, header=True)` 처럼 포맷별 단축 메서드도 있습니다. 경로에는 디렉터리, 와일드카드(`/logs/2026-09-*/`), 경로 리스트를 줄 수 있습니다.

4.2 예제 데이터 준비

```
import os, shutil, glob
from pyspark.sql import functions as F

base = "/tmp/book_io"
shutil.rmtree(base, ignore_errors=True)
os.makedirs(base)

csv_text = """order_id,user,amount,ts
1,kim,12000,2026-09-01 10:00:00
2,lee,8000,2026-09-01 11:30:00
3,kim,oops,2026-09-02 09:10:00
4,park,15000,2026-09-02 14:00:00
"""

with open(f"{base}/orders.csv", "w") as f:
    f.write(csv_text)
```

4.3 CSV: 가장 흔하고 가장 위험한

```
df = (spark.read
      .option("header", True)
      .option("inferSchema", True)
      .csv(f"{base}/orders.csv"))
df.printSchema()
```

출력

root

```
|-- order_id: integer (nullable = true)
|-- user: string (nullable = true)
|-- amount: string (nullable = true)
|-- ts: timestamp (nullable = true)
```

amount 에 `oops` 가 섞여 있어 전체가 `string` 으로 추론되었습니다. `inferSchema` 는 이런 문제 외에도 **파일을 한**

번 더 전부 읽어야 하므로 느립니다. 운영 코드에서는 스키마를 명시하십시오.

```
schema = ("order_id INT, user STRING, amount INT, "
          "ts TIMESTAMP, _corrupt_record STRING")
df = (spark.read
      .option("header", True)
      .option("mode", "PERMISSIVE")
      .option("columnNameOfCorruptRecord",
              "_corrupt_record")
      .schema(schema)
      .csv(f"{base}/orders.csv"))
df.show(truncate=False)
```

출력

order_id	user	amount	ts	_corrupt_record
1	kim	12000	2026-09-01 10:00:00	NULL
2	lee	8000	2026-09-01 11:30:00	NULL
3	kim	NULL	2026-09-02 09:10:00	3,kim,oops,2026-09-02 09:10:00
4	park	15000	2026-09-02 14:00:00	NULL

잘못된 행을 어떻게 다루지는 mode 로 정합니다.

mode	동작
PERMISSIVE (기본)	파싱 실패 필드는 null, 원문은 _corrupt_record (스키마에 있을 때)에
DROPMALFORMED	잘못된 행을 버림
FAILFAST	즉시 예외

자주 쓰는 CSV 옵션: sep, quote, escape, multiLine, nullValue, dateFormat, timestampFormat, encoding.

4.4 JSON

기본은 **한 줄에 JSON 객체 하나**(JSON Lines)입니다. 파일 전체가 하나의 배열이거나 객체가 여러 줄에 걸쳐 있으면 `multiLine=True` 가 필요합니다. 중첩 구조는 자동으로 `struct/array` 가 됩니다.

```
with open(f"{base}/events.json", "w") as f:
    f.write('{"id":1,"user":{"name":"kim",'
            '"tier":"gold"},"tags":["a","b"]}\n')
    f.write('{"id":2,"user":{"name":"lee",'
            '"tier":"basic"},"tags":[]}\n')
ev = spark.read.json(f"{base}/events.json")
ev.printSchema()
ev.select("id", "user.name",
          F.size("tags").alias("n")).show()
```

출력

root

|-- id: long (nullable = true)

|-- tags: array (nullable = true)

| |-- element: string (containsNull = true)

|-- user: struct (nullable = true)

| |-- name: string (nullable = true)

| |-- tier: string (nullable = true)

+---+-----+---+

| id|name| n|

+---+-----+---+

| 1| kim| 2|

| 2| lee| 0|

+---+-----+---+

중첩 필드는 `"user.name"` 처럼 점으로 접근합니다.

4.5 Parquet: 기본값으로 삼을 포맷

Parquet는 **컬럼 지향** 바이너리 포맷입니다. Spark 생태계의 사실상 표준이고, 특별한 이유가 없으면 중간·최종 결과를 모두 Parquet(또는 이를 기반으로 한 Delta/Iceberg)로 저장하십시오.

- **스키마 내장**: 타입 추론이 필요 없습니다.
- **컬럼 프루닝**: 필요한 컬럼만 디스크에서 읽습니다.
- **프레디킷 푸시다운**: 파일·row group마다 min/max 통계가 있어, `WHERE amount > 10000` 에 해당하지 않는 블록은 건너뜁니다.
- **압축**: 기본 snappy. CSV보다 보통 수 배 작습니다.

```
clean = df.filter(F.col("_corrupt_record").isNull()) \
    .drop("_corrupt_record") \
    .withColumn("dt", F.to_date("ts"))
(clean.write
    .mode("overwrite")
    .partitionBy("dt")
    .parquet(f"{base}/orders_pq"))

for p in sorted(glob.glob(f"{base}/orders_pq/**/*"),
                recursive=True)):
    if not os.path.basename(p).startswith("."):
        print(p.replace(base, ""))
```

출력

```
/orders_pq/_SUCCESS
/orders_pq/dt=2026-09-01
/orders_pq/dt=2026-09-01/part-00000-a9eabb06-665d-4b4d-b5c3-6c389c
bf9b9a.c000.snappy.parquet
/orders_pq/dt=2026-09-02
/orders_pq/dt=2026-09-02/part-00000-a9eabb06-665d-4b4d-b5c3-6c389c
bf9b9a.c000.snappy.parquet
```

출력을 보면 두 가지를 알 수 있습니다.

1. 결과는 파일이 아니라 디렉터리입니다. 각 Task가 자기 파티션을 `part-xxxxx` 파일로 따로 씁니다. 파일 하나를 원한다고 `coalesce(1)` 을 쓰면 모든 데이터가 한 Task로 몰리니, 작은 결과에만 쓰십시오.
2. `partitionBy("dt")` 는 `dt=값/ 하위 디렉터리를 만듭니다.` 이것이 Hive 스타일 파티셔닝입니다.

```
pq = spark.read.parquet(f"{base}/orders_pq")
q = pq.filter(F.col("dt") == "2026-09-02") \
    .select("user", "amount")
q.show()
q.explain()
```

출력

```
+-----+
|user|amount|
+-----+
|park| 15000|
+-----+
```

== Physical Plan ==

*(1) Project [user#80, amount#81]

+ - *(1) ColumnarToRow

+ - FileScan parquet [user#80,amount#81,dt#83] Batched: true, DataFilters: [], Format: Parquet, Location: InMemoryFileIndex(1 paths)[file:/tmp/book_io/orders_pq], PartitionFilters: [is notnull(dt#83), (dt#83 = 2026-09-02)], PushedFilters: [], ReadSchema: struct<user:string,amount:int>

계획의 `PartitionFilters` 는 `dt=2026-09-01` 디렉터리를 **아예 열지 않는**다는 뜻입니다(파티션 프루닝). `ReadSchema` 에는 `user`, `amount` 만 있습니다(컬럼 프루닝).

TIP

파티션 컬럼 고르기. 필터에 자주 쓰이고 카디널리티가 낮은 컬럼(날짜, 지역)이 좋습니다. `user_id` 처럼 값이 수백만 개인 컬럼으로 파티셔닝하면 작은 파일 수백만 개가 생겨 오히려 재앙이 됩니다. 파티션 하나에 최소 수백 MB가 쌓이는 수준이 적당합니다.

4.6 쓰기 모드

mode	대상이 이미 있으면
<code>errorifexists</code> (기본)	예외
<code>overwrite</code>	전부 지우고 씀
<code>append</code>	추가
<code>ignore</code>	아무것도 안 함

`partitionBy` 와 `overwrite` 를 함께 쓰면 기본적으로 **대상 디렉터리 전체**를 지웁니다. 특정 날짜 파티션만 갈아끼우려면 동적 파티션 덮어쓰기를 켜십시오.

```
spark.conf.set(
    "spark.sql.sources.partitionOverwriteMode",
    "dynamic")
daily.write.mode("overwrite").partitionBy("dt") \
    .parquet(path)  # daily에 있는 dt만 교체
```

4.7 테이블로 저장하기

경로 대신 **카탈로그 테이블**로 저장하면 이름으로 조회할 수

있고, 스키마·파티션 정보가 메타스토어에 기록됩니다.

```
spark.sql("DROP TABLE IF EXISTS orders")
clean.write.mode("overwrite").saveAsTable("orders")
spark.table("orders").groupBy("user").count().show()
```

출력

```
+----+-----+
|user|count|
+----+-----+
|park|    1|
| kim|    1|
| lee|    1|
+----+-----+
```

`insertInto("orders")` 는 기존 테이블에 **컬럼 이름이 아니라 위치 순서**로 넣습니다. 컬럼 순서가 다르면 조용히 값이 뒤섞이니 주의하십시오.

4.8 테이블 포맷: Delta Lake, Iceberg

Parquet 디렉터리는 편하지만 **트랜잭션이 없습니다**. 쓰기 도중 실패하면 반쯤 쓴 파일이 남고, 동시에 읽는 쪽은 깨진 상태를 봅니다. `UPDATE` · `MERGE` 도 없습니다.

Delta Lake와 Apache Iceberg는 Parquet 파일 위에 **트랜잭션 로그**를 얹어 이 문제를 푼니다.

- ACID 쓰기, 스냅샷 격리
- `MERGE INTO (upsert)`, `UPDATE`, `DELETE`
- 타임 트래블(`VERSION AS OF`)
- 스키마 진화, 파일 컴팩션

```
df.write.format("delta").mode("append").save(path)
spark.read.format("iceberg").load("catalog.db.tbl")
```

별도 패키지 설정이 필요하므로 이 책에서는 개념만 소개합니다. 새로 데이터 레이크를 설계한다면 둘 중 하나를 기본으로 삼으십시오.

4.9 JDBC: 데이터베이스 읽기

```
orders = (spark.read.format("jdbc")
    .option("url", "jdbc:postgresql://db:5432/shop")
    .option("dbtable", "public.orders")
    .option("user", user).option("password", pw)
    .option("partitionColumn", "order_id")
    .option("lowerBound", 1)
    .option("upperBound", 10_000_000)
    .option("numPartitions", 16)
    .option("fetchsize", 10_000)
    .load())
```

`partitionColumn`·`경계`·`numPartitions`를 주지 않으면 **단일 커넥션, 단일 Task**로 전체 테이블을 읽습니다. 분산 처리의 의미가 없어지지요. 위 설정은 `order_id` 범위를 16등분해 16개 쿼리를 병렬로 날립니다. 경계값은 필터가 아니라 분할 기준일 뿐이라 범위 밖 행도 첫/마지막 파티션에 포함됩니다. 원본 DB가 감당할 수 있는 동시 커넥션 수를 넘기지 않도록 `numPartitions`를 정하십시오.

`dbtable` 대신 `query` 옵션으로 서브쿼리를 줄 수도 있고, 이때 필터는 DB 쪽에서 실행됩니다.

4.10 핵심 정리

- CSV/JSON은 스키마를 명시하고, `mode` 로 불량 행 정책을 정하십시오.
- 저장은 Parquet(가능하면 Delta/Iceberg). 컬럼·파티션 프루닝의 혜택을 받습니다.
- 출력은 디렉터리입니다. `partitionBy` 는 저카디널리티 필터 컬럼으로.
- JDBC는 `partitionColumn` 을 주지 않으면 병렬이 아닙니다.

내장 함수로 변환하기

`pyspark.sql.functions` (이하 `F`)에는 수백 개의 함수가 있습니다. 전부 JVM에서 실행되는 **네이티브 함수**라 빠릅니다. Python UDF를 쓰기 전에 반드시 여기서 먼저 찾으십시오. 거의 항상 있습니다.

```
from pyspark.sql import functions as F

users = spark.createDataFrame([
    (1, "Kim Minsu ", "kim@a.com", "2026-01-15", None),
    (2, "lee jiwon", "LEE@b.co.kr", "2025-11-03", "VIP"),
    (3, None, "park@c.com", "2026-08-30", "basic"),
], "id INT, name STRING, email STRING, "
    "joined STRING, tier STRING")
```

5.1 조건식: when / otherwise

pandas의 `np.where`, SQL의 `CASE WHEN` 에 해당합니다.

```
users.select(
    "id", "tier",
    F.when(F.col("tier") == "VIP", 3)
      .when(F.col("tier") == "basic", 1)
      .otherwise(0).alias("level"),
).show()
```

출력

id	tier	level
1	NULL	0
2	VIP	3
3	basic	1

`otherwise` 를 빼면 어느 조건에도 맞지 않는 행은 `null` 입니다.
`tier` 가 `null` 인 1번 행은 `== "VIP"` 의 결과가 `null` (참도 거짓도 아님)이라 `otherwise` 로 떨어졌습니다.

5.2 null 다루기

SQL과 같은 **3값 논리**를 따릅니다. `null` 과의 비교는 `null` 이고, `filter` 는 `null` 을 거짓으로 취급합니다.

```
print(users.filter(F.col("tier") != "VIP").count())
print(users.filter(~F.col("tier").eqNullSafe("VIP"))
      .count())
```

출력

1
2

`!= "VIP"` 는 `tier` 가 `null` 인 행을 놓칩니다. 이런 실수가 조용한 데이터 누락의 흔한 원인입니다. `null` 까지 같은 값으로 비교하려면 `eqNullSafe` (SQL의 `<=>`)를 쓰십시오.

```
users.select(
  "id",
  F.coalesce("tier", F.lit("none")).alias("tier"),
  F.col("name").isNull().alias("no_name"),
).show()

users.na.fill({"tier": "none", "name": "?"}).show()
users.na.drop(subset=["name"]).count()
```

출력

id	tier	no_name
1	none	false
2	VIP	false
3	basic	true

id	name	email	joined	tier
1	Kim Minsu	kim@a.com	2026-01-15	none
2	lee jiwon	LEE@b.co.kr	2025-11-03	VIP
3	?	park@c.com	2026-08-30	basic

- `F.coalesce(a, b, ...)` : 처음으로 `null`이 아닌 값.
- `df.na.fill(값 또는 dict)`, `df.na.drop(how, subset)` : pandas의 `fillna`, `dropna`.
- `F.isnanv` : 부동소수점 `NaN` 처리. Spark에서 `NaN` 과 `null` 은 다른 값입니다. pandas에서 넘어온 데이터라면 둘 다 확인하십시오.

5.3 문자열

```
users.select(
    F.initcap(F.trim("name")).alias("name"),
    F.lower("email").alias("email"),
    F.split("email", "@").getItem(1).alias("domain"),
    F.regexp_extract("email", r"^([^\@]+)\@", 1)
    .alias("local"),
    F.regexp_replace("email", r"\.co\.kr$", ".kr")
    .alias("r"),
).show(truncate=False)
```

출력

name	email	domain	local	r
Kim Minsu	kim@a.com	a.com	kim	kim@a.com
Lee Jiwon	lee@b.co.kr	b.co.kr	LEE	LEE@b.kr
NULL	park@c.com	c.com	park	park@c.com

함수	용도
length, upper, lower, initcap, trim / ltrim / rtrim	기본
substring(col, pos, len)	1부터 시작하는 부분 문자열
concat, concat_ws(sep, ...)	연결 (concat 은 하나라도 null이면 null)
split(col, regex)	배열로 분리
regexp_extract, regexp_replace, rlike	정규식
like, ilike, startswith, endswith, contains	Column 메서드
lpad, rpad, format_string, translate	서식

`F.col("email").rlike("@c\\\.com$")` 처럼 Column 메서드로 쓰는 것들도 있습니다.

5.4 날짜와 시간

```
d = users.select(
    "joined",
    F.to_date("joined").alias("dt"),
).withColumns({
    "year": F.year("dt"),
    "dow": F.dayofweek("dt"),          # 1=일요일
    "month_start": F.trunc("dt", "month"),
    "plus_30": F.date_add("dt", 30),
    "days": F.datediff(F.lit("2026-09-21"), "dt"),
    "fmt": F.date_format("dt", "yyyy/MM/dd E"),
})
d.show()
```

출력

joined	dt	year	dow	month_start	plus_30	days	fmt
2026-01-15	2026-01-15	2026	5	2026-01-01	2026-02-14	249	2026/01/15 Thu
2025-11-03	2025-11-03	2025	2	2025-11-01	2025-12-03	322	2025/11/03 Mon
2026-08-30	2026-08-30	2026	1	2026-08-01	2026-09-29	22	2026/08/30 Sun

- 파싱: `to_date(col, fmt)`, `to_timestamp(col, fmt)`. 형식이 맞지 않으면 ANSI 모드에서는 **예외**입니다. 지저분한 입력에는 `try_to_timestamp` 를 쓰십시오.
- 서식 패턴은 Java `DateTimeFormatter` 문법입니다(`yyyy-MM-dd HH:mm:ss`). pandas의 `%Y-%m-%d` 와 다르니 주의하십시오.
- 절단: 날짜는 `trunc(col, "month")`, 타임스탬프는 `date_trunc("hour", col)`.

- 현재 시각: `current_date()`, `current_timestamp()`. 쿼리 하나 안에서는 모든 행이 같은 값을 받습니다.
- Unix 시간: `unix_timestamp`, `from_unixtime`, `timestamp_seconds`.

5.5 배열, 맵, 구조체

Spark는 중첩 타입을 1급으로 다룹니다. 정규화하지 않은 로그·JSON을 다룰 때 매우 유용합니다.

```
orders = spark.createDataFrame([
    (1, ["apple", "pear"], {"coupon": "Y"}),
    (2, ["kiwi"],          {}),
    (3, [],                {"coupon": "N"}),
], "oid INT, items ARRAY<STRING>, "
  "meta MAP<STRING,STRING>")

orders.select(
    "oid",
    F.size("items").alias("n"),
    F.array_contains("items", "kiwi").alias("has_kiwi"),
    F.get("items", 0).alias("first"),
    F.col("meta")["coupon"].alias("coupon"),
).show()
```

출력

oid	n	has_kiwi	first	coupon
1	2	false	apple	Y
2	1	true	kiwi	NULL
3	0	false	NULL	N

주의

`F.col("items")[0]` 도 쓸 수 있지만, ANSI 모드(4.x 기본)에서는 **빈 배열에 대해 `INVALID_ARRAY_INDEX` 예외**가 납니다. 범위를 벗어나면 `null`을 받고 싶을 때는 `F.get(col, i)` (0부터) 또는 `F.try_element_at(col, i)` (1부터)를 쓰십시오.

explode: 배열을 행으로

```
orders.select("oid",  
              F.explode("items").alias("item")).show()
```

출력

```
+---+-----+  
|oid| item|  
+---+-----+  
|  1|apple|  
|  1| pear|  
|  2| kiwi|  
+---+-----+
```

3번 주문이 사라졌습니다. 빈 배열·`null`은 `explode` 에서 행을 만들지 않습니다. 남기려면 `explode_outer` 를 쓰십시오. 인덱스가 필요하다면 `posexplode` 입니다.

반대 방향(행 → 배열)은 집계 함수 `collect_list` / `collect_set` 입니다(6장).

구조체

```
s = orders.select(
  "oid",
  F.struct(
    F.size("items").alias("n"),
    F.get("items", 0).alias("first"),
  ).alias("info"),
)
s.printSchema()
s.select("oid", "info.*").show()
```

출력

root

```
|-- oid: integer (nullable = true)
|-- info: struct (nullable = false)
|   |-- n: integer (nullable = true)
|   |-- first: string (nullable = true)
```

```
+---+---+-----+
|oid|  n|first|
+---+---+-----+
|  1|  2|apple|
|  2|  1| kiwi|
|  3|  0| NULL|
+---+---+-----+
```

"info.*" 는 구조체의 필드를 최상위 컬럼으로 펼칩니다.

고차 함수: 배열을 explode 없이 처리

배열 원소마다 계산하려고 explode → 가공 → collect_list
를 하면 셔플이 생깁니다. 고차 함수를 쓰면 행 안에서

끝납니다.

```
nums = spark.createDataFrame(
    [[(1, 2, 3, 4),], ([10, 15],)], "xs ARRAY<INT>")
nums.select(
    F.transform("xs", lambda x: x * 10).alias("x10"),
    F.filter("xs", lambda x: x % 2 == 0).alias("even"),
    F.aggregate("xs", F.lit(0), lambda acc, x: acc + x)
    .alias("sum"),
    F.exists("xs", lambda x: x > 12).alias("any_gt12"),
).show()
```

출력

x10	even	sum	any_gt12
[10, 20, 30, 40]	[2, 4]	10	false
[100, 150]	[10]	25	true

여기 `lambda` 는 Python 함수처럼 보이지만 **Python에서 실행되지 않습니다**. 인자로 받은 Column 객체로 표현식을 한번 조립할 뿐이고, 실행은 JVM이 합니다. 그래서 UDF와 달리 빠릅니다.

5.6 JSON 문자열 다루기

컬럼 안에 JSON 문자열이 들어 있는 경우가 많습니다.

```
raw = spark.createDataFrame(
    [({'a': 1, "b": {'c': "x"}}),
      ({'a': 2, "b": {'c': "y"}})], "js STRING")
parsed = raw.select(
    F.from_json("js", "a INT, b STRUCT<c: STRING>")
    .alias("j"))
parsed.select("j.a", "j.b.c").show()
raw.select(F.get_json_object("js", "$.b.c")
    .alias("c")).show()
```

출력

```
+---+---+
| a | c |
+---+---+
| 1 | x |
| 2 | y |
+---+---+
```

```
+---+
| c |
+---+
| x |
| y |
+---+
```

- `from_json(col, schema)` : 스키마대로 파싱해 struct로. 여러 필드를 쓸 때 효율적입니다.
- `get_json_object(col, path)` : 필드 하나를 문자열로 뽑을 때.
- `to_json(struct)` : 반대 방향.
- `schema_of_json(샘플)` : 스키마 문자열 추론 도우미.

NOTE

VARIANT 타입 (4.0+). 스키마가 제각각인 JSON을 매번 `from_json` 으로 파싱하는 대신, `parse_json` 으로 바이너리 반정형 값으로 한 번 변환해 저장하고 `variant_get(v, "$.b.c", "string")` 으로 꺼내 쓸 수 있습니다. 문자열 JSON보다 조회가 훨씬 빠릅니다.

```
v = raw.select(F.parse_json("js").alias("v"))
v.select(
    F.variant_get("v", "$.a", "int").alias("a"),
    F.variant_get("v", "$.b.c", "string").alias("c"),
).show()
```

출력

```
+---+---+
| a | c |
+---+---+
| 1 | x |
| 2 | y |
+---+---+
```

5.7 핵심 정리

- UDF 전에 `F` 를 먼저 뒤지십시오. 거의 다 있습니다.
- `null` 은 3값 논리. `!=` 필터가 `null` 행을 조용히 버립니다.
- 날짜 형식은 Java 패턴(`yyyy-MM-dd`), 지저분한 입력은 `try_*` .
- 배열은 `explode` (행으로) 또는 `고차 함수`(행 안에서).

집계와 조인

6.1 예제 데이터

```
from pyspark.sql import functions as F

sales = spark.createDataFrame([
    ("2026-09-01", "Seoul", "kim", "book", 12000),
    ("2026-09-01", "Seoul", "lee", "food", 8000),
    ("2026-09-01", "Busan", "park", "book", 15000),
    ("2026-09-02", "Seoul", "kim", "food", 6000),
    ("2026-09-02", "Busan", "choi", "toy", 30000),
    ("2026-09-02", "Busan", "park", "book", 9000),
    ("2026-09-03", "Daegu", "jung", "food", 4000),
], "dt STRING, city STRING, user STRING, "
    "cat STRING, amount INT")
```

6.2 groupBy + agg

```
(sales.groupBy("city")
  .agg(F.count("*").alias("n"),
       F.sum("amount").alias("total"),
       F.round(F.avg("amount"), 1).alias("avg"),
       F.countDistinct("user").alias("users"),
       F.max_by("user", "amount").alias("top_user"))
  .orderBy(F.desc("total"))
  .show())
```

출력

city	n	total	avg	users	top_user
Busan	3	54000	18000.0	2	choi
Seoul	3	26000	8666.7	2	kim
Daegu	1	4000	4000.0	1	jung

pandas의 `groupby().agg()` 와 비슷하지만, 결과 컬럼 이름을 `alias` 로 붙이는 방식입니다. 이름을 안 붙이면 `sum(amount)` 같은 이름이 생겨 다루기 불편합니다.

자주 쓰는 집계 함수

함수	설명
<code>count("*")</code> / <code>count(col)</code>	전체 행 / null 아닌 값 수
<code>sum</code> , <code>avg</code> / <code>mean</code> , <code>min</code> , <code>max</code>	기본
<code>countDistinct</code> / <code>count_distinct</code>	정확한 고유값 수 (비쌘)
<code>approx_count_distinct(col, rsd)</code>	HyperLogLog 근사. 대용량에서 권장
<code>collect_list</code> / <code>collect_set</code>	그룹 값을 배열로
<code>first</code> , <code>last</code> , <code>any_value</code>	순서 보장 없음

함수	설명
<code>max_by(x, y), min_by(x, y)</code>	y가 최대/최소인 행의 x
<code>percentile_approx(col, p), median</code>	분위수
<code>stddev, variance, corr, skewness</code>	통계
<code>sum(F.when(cond, 1).otherwise(0))</code>	조건부 카운트 (<code>count_if</code> 도 있음)

부분 집계: `groupBy`가 생각보다 싼 이유

2장의 `explain()` 에서 `HashAggregate` 가 셔플 전후로 두 번 나온 것을 기억하십니까? Spark는 셔플 **전에** 각 파티션에서 부분합(`partial`)을 먼저 계산하고, 셔플로는 그 부분합만 보냅니다. MapReduce의 `combiner`와 같은 아이디어입니다. 그래서 `sum · count · max` 는 원본 행 수와 무관하게 셔플량이 **그룹 수 × 파티션 수** 정도로 작습니다.

반면 `collect_list` 와 정확한 `countDistinct` 는 값 자체를 셔플해야 해서 비쌉니다. 그룹 하나가 거대하면 한 Task의 메모리를 넘칠 수도 있습니다.

전체 집계와 다중 키

```
sales.agg(F.sum("amount").alias("total"),
          F.min("dt"), F.max("dt")).show()
sales.groupBy("city", "cat").sum("amount") \
      .orderBy("city", "cat").show()
```

출력

```
+-----+-----+-----+
|total|  min(dt)|  max(dt)|
+-----+-----+-----+
|84000|2026-09-01|2026-09-03|
+-----+-----+-----+
```

```
+-----+-----+-----+
| city| cat|sum(amount)|
+-----+-----+-----+
|Busan|book|      24000|
|Busan|toy |      30000|
|Daegu|food|       4000|
|Seoul|book|      12000|
|Seoul|food|      14000|
+-----+-----+-----+
```

pivot

```
(sales.groupBy("city")
      .pivot("cat", ["book", "food", "toy"])
      .sum("amount")
      .na.fill(0)
      .show())
```

출력

```
+-----+-----+-----+
| city| book| food| toy|
+-----+-----+-----+
|Daegu|    0| 4000|  0|
|Busan|24000|    0|30000|
|Seoul|12000|14000|  0|
+-----+-----+-----+
```

`pivot` 에 값 목록을 주지 않으면 Spark가 고유값을 구하려고 작업을 한 번 더 실행합니다. 값을 알고 있다면 넘겨주십시오.

rollup / cube: 소계 만들기

```
(sales.rollup("city", "cat")
  .agg(F.sum("amount").alias("total"))
  .orderBy(F.col("city").asc_nulls_last(),
           F.col("cat").asc_nulls_last())
  .show())
```

출력

city	cat	total
Busan	book	24000
Busan	toy	30000
Busan	NULL	54000
Daegu	food	4000
Daegu	NULL	4000
Seoul	book	12000
Seoul	food	14000
Seoul	NULL	26000
NULL	NULL	84000

`null` 이 “전체”를 뜻합니다. 원래 데이터에 `null` 이 있다면 `F.grouping("city")` 로 구분하십시오. `cube` 는 모든 조합의 소계를 만듭니다.

6.3 조인

```
users = spark.createDataFrame([
  ("kim", "gold"), ("lee", "basic"),
  ("park", "gold"), ("yoon", "basic"),
], "user STRING, tier STRING")
```

조인 종류

```
j = sales.join(users, on="user", how="left")
j.select("user", "amount", "tier").show()
```

출력

```
+---+-----+---+
|user|amount| tier|
+---+-----+---+
| kim| 12000| gold|
| park| 15000| gold|
| lee| 8000| basic|
| kim| 6000| gold|
| choi| 30000| NULL|
| park| 9000| gold|
| jung| 4000| NULL|
+---+-----+---+
```

how	결과
inner (기본)	양쪽에 모두 있는 키
left / right	한쪽 전체 + 매칭 (없으면 null)
full / outer	양쪽 전체
left_semi	오른쪽에 있는 왼쪽 행만 (왼쪽 컬럼만)
left_anti	오른쪽에 없는 왼쪽 행만
cross	카티션 곱 (crossJoin)

left_semi 와 left_anti 는 SQL의 WHERE EXISTS , WHERE NOT EXISTS 입니다. “오른쪽에 있는지”만 확인하므로, inner 조인

후 `dropDuplicates` 하는 것보다 싸고 행이 뺄뵈기될 걱정도 없습니다.

```
# 등급 정보가 없는 구매자
sales.join(users, "user", "left_anti") \
      .select("user").distinct().show()
```

출력

```
+----+
|user|
+----+
|choi|
|jung|
+----+
```

조인 조건 쓰기

- **on="key" 또는 on=["k1", "k2"]** : 이름이 같은 컬럼으로 조인. 결과에 키 컬럼이 **하나만** 남습니다. 가장 깔끔합니다.
- **표현식**: 이름이 다르거나 조건이 복잡할 때. 이름이 같으면 양쪽 컬럼이 **둘 다** 남아 이후 참조가 모호해집니다.

```

a = sales.alias("a")
b = users.alias("b")
x = a.join(b, F.col("a.user") == F.col("b.user"),
          "inner")
print(x.columns)
x.select("a.user", "b.tier", "a.amount").show(3)

```

출력

```
['dt', 'city', 'user', 'cat', 'amount', 'user', 'tier']
```

```
+---+-----+-----+
```

```
|user| tier|amount|
```

```
+---+-----+-----+
```

```
| kim| gold| 12000|
```

```
|park| gold| 15000|
```

```
| lee|basic| 8000|
```

```
+---+-----+-----+
```

```
only showing top 3 rows
```

alias 를 붙여 두고 "a.user" 처럼 한정해서 참조하는 것이 가장 안전합니다. 비등가 조인(범위 조인)도 표현식으로 씁니다: (a.ts >= b.start) & (a.ts < b.end) .

주의

조인 키의 null 은 서로 매칭되지 않습니다. SQL과 같습니다.

null 끼리 맞추려면 a.k.eqNullSafe(b.k) 를 조건으로

쓰십시오. 또 키가 양쪽에서 중복되면 행이 곱으로

들어옵니다. 조인 후 행 수가 예상과 다르면 키의 고유성부터

확인하십시오: df.groupBy("k").count().filter("count > 1") .

6.4 조인 전략: Spark는 어떻게 조인하는가

분산 환경에서 조인은 같은 키의 행을 같은 곳에 모으는 문제입니다. Spark의 선택지는 크게 둘입니다.

Broadcast Hash Join (BHJ). 한쪽이 작으면 그 테이블 전체를 모든 **Executor**에 복사합니다. 큰 쪽은 움직이지 않고 제자리에서 해시 조회만 하면 됩니다. **셔플이 없으므로 가장 빠릅니다.** 기본적으로 한쪽 추정 크기가 `spark.sql.autoBroadcastJoinThreshold` (기본 10MB) 이하면 자동 선택됩니다.

Sort Merge Join (SMJ). 둘 다 크면 양쪽을 키로 **셔플**하고, 파티션 안에서 정렬해 병합합니다. 셔플 두 번 + 정렬이라 비싸지만 메모리 걱정 없이 어떤 크기든 처리합니다. 큰 테이블끼리의 기본 전략입니다.

(키 쪽 파티션이 메모리에 들어갈 만하면 정렬 없이 해시 테이블을 만드는 Shuffle Hash Join도 있습니다.)

```
names = F.array(*[F.lit(u) for u in
                  ["kim", "lee", "park", "yoon"]])
big = spark.range(1_000_000).withColumn(
    "user", F.get(names, (F.col("id") % 4).cast("int")))
j = big.join(users, "user")
j.explain()
```

출력

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Project [user#312, id#311L, tier#286]
   +- SortMergeJoin [user#312], [user#285], Inner
      :- Sort [user#312 ASC NULLS FIRST], false, 0
         :- +- Exchange hashpartitioning(user#312, 8), ENSURE_REQUIREMENTS, [plan_id=831]
            : +- Project [id#311L, [kim,lee,park,yoon][cast((id#311L % 4) as int)] AS user#312]
            :    +- Filter isnotnull([kim,lee,park,yoon][cast((id#311L % 4) as int)])
            :    +- Range (0, 1000000, step=1, splits=4)
      +- Sort [user#285 ASC NULLS FIRST], false, 0
         +- Exchange hashpartitioning(user#285, 8), ENSURE_REQUIREMENTS, [plan_id=832]
            +- Filter isnotnull(user#285)
               +- Scan ExistingRDD[user#285,tier#286]
```

`users` 는 네 줄짜리인데도 `SortMergeJoin` 이 선택되었고, 양쪽에 `Exchange` (셔플)가 붙었습니다. 메모리의 Python 리스트로 만든 `DataFrame`(`Scan ExistingRDD`)은 **크기 통계가 없어서** Spark가 작다는 사실을 모르기 때문입니다. 복잡한

서브쿼리 결과도 마찬가지로 추정이 부정확합니다. 이럴 때는 힌트로 브로드캐스트를 강제합니다.

```
big.join(F.broadcast(users), "user").explain()
```

출력

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Project [user#312, id#311L, tier#286]
   +- BroadcastHashJoin [user#312], [user#285], Inner, BuildRight, false, false
      :- Project [id#311L, [kim,lee,park,yoon][cast((id#311L % 4) as int)] AS user#312]
         : +- Filter isnotnull([kim,lee,park,yoon][cast((id#311L % 4) as int)])
            : +- Range (0, 1000000, step=1, splits=4)
      +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, false]),false),
         [plan_id=865]
         +- Filter isnotnull(user#285)
            +- Scan ExistingRDD[user#285,tier#286]
```

이제 BroadcastHashJoin 이고, 큰 쪽(Range)에는 Exchange 가 없습니다. SQL에서는 SELECT /*+ BROADCAST(u) */ ... 힌트를 씁니다.

사실 힌트가 없어도 AQE(10장)가 켜져 있으면 실행 도중 셔플 결과의 **실제 크기**를 보고 전략을 바꿉니다. 첫 번째 j 를 실제로 실행한 뒤 계획을 보면 이렇게 바뀌어 있습니다.

```
AdaptiveSparkPlan isFinalPlan=true
+- == Final Plan ==
   +- BroadcastHashJoin ... BuildRight
      :- AQEShuffleRead local
         : +- ShuffleQueryStage 0 ...
      +- BroadcastQueryStage 2 ...
+- == Initial Plan ==
   +- SortMergeJoin ...
```

다만 AQE는 이미 한쪽을 셔플한 **뒤에** 바꾸는 것이라, 처음부터 브로드캐스트한 것보다는 비쌉니다. 작다는 것을 알면 힌트를 주는 편이 낫습니다.

TIP

브로드캐스트되는 작은 **Driver**를 거쳐 모든 Executor에 보내집니다. 수백 MB를 넘기면 Driver OOM이나 브로드캐스트 타임아웃이 납니다. “작은 쪽”의 기준은 대략 수십 MB, 크게 잡아도 수백 MB까지입니다. 또 `left` 조인에서는 **오른쪽만** 브로드캐스트할 수 있습니다(왼쪽을 복사하면 매칭 안 된 행이 중복 출력되므로).

6.5 union

```
d1 = spark.createDataFrame([(1, "a")],  
                           "id INT, v STRING")  
d2 = spark.createDataFrame([("b", 2)],  
                           "v STRING, id INT")  
d1.unionByName(d2).show()
```

출력

```
+---+---+  
| id|  v|  
+---+---+  
|  1|  a|  
|  2|  b|  
+---+---+
```

`union` 은 **위치 순서**로 붙입니다. 컬럼 순서가 다르면 에러 없이 값이 뒤섞이거나(타입이 호환되면) 이상한 캐스트가 일어납니다. 항상 `unionByName` 을 쓰십시오. 한쪽에 없는 컬럼은 `allowMissingColumns=True` 로 `null` 채움이 됩니다. `union` 은 SQL의 `UNION ALL` 처럼 중복을 제거하지 않습니다. 필요하면 뒤에 `distinct()` 를 붙이십시오.

6.6 핵심 정리

- 집계 결과 컬럼은 `alias` 로 이름 붙이기. `sum / count` 는 부분 집계로 씁니다.
- `left_semi / left_anti` 는 존재 여부 필터의 정석입니다.
- 작은 쪽은 브로드캐스트(셔플 없음), 큰 쪽끼리는 Sort Merge(셔플 2회).
- 조인 키 중복 = 행 폭발. `null` 키는 매칭 안 됨.
- `union` 대신 `unionByName` .

CHAPTER 7

윈도 함수

`groupBy` 는 그룹당 한 행으로 줄입니다. 윈도 함수는 행 수를 유지한 채 각 행 옆에 “그룹 안에서의 계산 결과”를 붙입니다. `pandas`의 `groupby().transform()`, `rank()`, `shift()`, `rolling()`, `cumsum()` 이 전부 여기에 해당합니다.

```
from pyspark.sql import functions as F
from pyspark.sql.window import Window

ev = spark.createDataFrame([
    ("kim", "2026-09-01 10:00", 100),
    ("kim", "2026-09-01 10:20", 300),
    ("kim", "2026-09-01 12:00", 200),
    ("lee", "2026-09-01 09:00", 500),
    ("lee", "2026-09-01 09:05", 500),
    ("lee", "2026-09-02 18:00", 100),
], "user STRING, ts STRING, amount INT") \
    .withColumn("ts", F.to_timestamp("ts"))
```

7.1 윈도 정의

윈도는 세 요소로 정의합니다.

1. **partitionBy**: 어떤 행끼리 묶을지 (`groupBy`의 키)
2. **orderBy**: 묶음 안의 순서
3. **frame**: 현재 행 기준으로 어디부터 어디까지 볼지

```
w = Window.partitionBy("user").orderBy("ts")
```

7.2 순위

```
ev.select(
    "user", "amount",
    F.row_number().over(
        Window.partitionBy("user")
                .orderBy(F.desc("amount"))).alias("rn"),
    F.rank().over(
        Window.partitionBy("user")
                .orderBy(F.desc("amount"))).alias("rank"),
    F.dense_rank().over(
        Window.partitionBy("user")
                .orderBy(F.desc("amount"))).alias("dense"),
).show()
```

출력

user	amount	rn	rank	dense
kim	300	1	1	1
kim	200	2	2	2
kim	100	3	3	3
lee	500	1	1	1
lee	500	2	1	1
lee	100	3	3	2

동점(lee의 500, 500)에서 차이가 납니다. `row_number` 는 동점이어도 1, 2로 **임의로** 갈라 줍니다. 결정적인 결과가 필요하면 `orderBy` 에 타이브레이커 컬럼을 추가하십시오.

가장 흔한 용도: 키별 최신 1건

```
latest = (ev
    .withColumn("rn", F.row_number().over(
        Window.partitionBy("user")
            .orderBy(F.desc("ts"))))
    .filter("rn = 1")
    .drop("rn"))
latest.show()
```

출력

user	ts	amount
kim	2026-09-01 12:00:00	200
lee	2026-09-02 18:00:00	100

CDC 로그에서 최신 상태 뽑기, 중복 레코드 정리 등에 쓰는 표준 패턴입니다. `dropDuplicates` 와 달리 어떤 행이 남을지 통제할 수 있습니다.

7.3 이전·다음 행: lag / lead

```
ev.select(
  "user", "ts",
  F.lag("ts").over(w).alias("prev_ts"),
  ((F.col("ts").cast("long")
    - F.lag("ts").over(w).cast("long")) / 60)
    .alias("gap_min"),
  F.lead("amount", 1, 0).over(w).alias("next_amt"),
).show()
```

출력

user	ts	prev_ts	gap_min	next_amt
kim	2026-09-01 10:00:00	NULL	NULL	300
kim	2026-09-01 10:20:00	2026-09-01 10:00:00	20.0	200
kim	2026-09-01 12:00:00	2026-09-01 10:20:00	100.0	0
lee	2026-09-01 09:00:00	NULL	NULL	500
lee	2026-09-01 09:05:00	2026-09-01 09:00:00	5.0	100
lee	2026-09-02 18:00:00	2026-09-01 09:05:00	1975.0	0

`lag(col, n, default)`: n행 앞의 값. 첫 행은 `default` (기본 `null`)입니다.

7.4 누적과 이동 집계: 프레임

집계 함수에 `.over(w)` 를 붙이면 윈도우 집계입니다. 이때

프레임이 중요합니다.

```
running = w.rowsBetween(Window.unboundedPreceding,
                          Window.currentRow)
moving2 = w.rowsBetween(-1, Window.currentRow)
whole = Window.partitionBy("user")

ev.select(
    "user", "amount",
    F.sum("amount").over(running).alias("cum"),
    F.avg("amount").over(moving2).alias("mv2"),
    F.sum("amount").over(whole).alias("user_total"),
    (F.col("amount") / F.sum("amount").over(whole))
      .alias("share"),
).show()
```

출력

user	amount	cum	mv2	user_total	share
kim	100	100	100.0	600	0.16666666666666666
kim	300	400	200.0	600	0.5
kim	200	600	250.0	600	0.3333333333333333
lee	500	500	500.0	1100	0.45454545454545453
lee	500	1000	500.0	1100	0.45454545454545453
lee	100	1100	300.0	1100	0.09090909090909091

- `rowsBetween(a, b)` : **행 개수** 기준. `-1` 은 한 행 앞.
- `rangeBetween(a, b)` : **orderBy 값** 기준. “현재 시각부터 1시간 전까지”처럼 값의 범위로 자를 때. `orderBy` 컬럼이 하나이고 숫자여야 하므로 타임스탬프는 `cast("long")` (초)로 바꿔 씁니다.
- `orderBy` 가 **없는** 윈도우 (`whole`) 는 파티션 전체가 프레임입니다. pandas의 `groupby().transform("sum")` 과 같습니다.

```
hour = (Window.partitionBy("user")
        .orderBy(F.col("ts").cast("long"))
        .rangeBetween(-3600, 0))
ev.select("user", "ts", "amount",
        F.sum("amount").over(hour).alias("last_1h")) \
    .show()
```

출력

user	ts	amount	last_1h
kim	2026-09-01 10:00:00	100	100
kim	2026-09-01 10:20:00	300	400
kim	2026-09-01 12:00:00	200	200
lee	2026-09-01 09:00:00	500	500
lee	2026-09-01 09:05:00	500	1000
lee	2026-09-02 18:00:00	100	100

주의

기본 프레임의 함정. `orderBy` 가 있는 윈도우에 프레임을 지정하지 않으면 기본값은 `rangeBetween(unboundedPreceding, currentRow)` 입니다. 그래서 `F.last("x").over(w)` 는 “파티션의 마지막 값”이 아니라 “**현재 행까지의 마지막 값**”, 즉 **자기 자신을 돌려줍니다**. 또 `range` 프레임이라 `orderBy` 값이 같은 행들(동점)은 한꺼번에 포함되어, 누적합이 계단처럼 될 수 있습니다. 누적합에는 `rowsBetween` 을 명시하는 습관을 들이십시오.

7.5 윈도우와 성능

윈도우 함수는 `partitionBy` 키로 **셔플**한 뒤 파티션 안에서 **정렬**합니다. 비용 면에서 `groupBy` 보다 무겁습니다.

- `partitionBy` 없이 `Window.orderBy(...)` 만 쓰면 **모든 데이터가 한 파티션으로** 모입니다. Spark가 경고(`No Partition Defined for Window operation!`)를 띄웁니다. 전역 순번이 꼭 필요하다면 `monotonically_increasing_id()` (연속적이지 않은 고유 ID)로 대체할 수 있는지 먼저 검토하십시오.
- 파티션 키 하나에 행이 몰리면(예: `user = 'guest'`) 그 Task만 한없이 느려집니다. 10장의 데이터 스큐를 보십시오.
- 같은 `partitionBy / orderBy` 를 쓰는 윈도우 함수 여러 개는 Spark가 한 번의 정렬로 묶어 처리합니다. 윈도우 정의를 변수로 재사용하면 이 최적화를 받기 쉽습니다.

7.6 시간 버킷 집계: `F.window`

이름이 비슷하지만 다른 기능입니다. 타임스탬프를 고정 크기 구간으로 잘라 `groupBy` 키로 씁니다. 스트리밍(12장)에서 특히

많이 씁니다.

```
(ev.groupBy("user", F.window("ts", "1 hour"))
  .agg(F.sum("amount").alias("amt"))
  .select("user", "window.start", "amt")
  .orderBy("user", "start")
  .show())
```

출력

user	start	amt
kim	2026-09-01 10:00:00	400
kim	2026-09-01 12:00:00	200
lee	2026-09-01 09:00:00	1000
lee	2026-09-02 18:00:00	100

`F.window(ts, "10 minutes", "5 minutes")` 처럼 슬라이드 간격을 주면 겹치는 구간(sliding window)이 됩니다. 활동 간격 기반의 세션 구간은 `F.session_window(ts, "30 minutes")` 입니다.

7.7 핵심 정리

- 윈도우 = `partitionBy` + `orderBy` + `frame`. 행 수가 유지됩니다.
- 키별 최신 1건: `row_number().over(...desc) → filter(rn == 1)`.
- 누적은 `rowsBetween` 을 명시. `last` 의 기본 프레임 함정 주의.
- `partitionBy` 없는 윈도우는 단일 파티션으로 몰립니다.

Spark SQL

2장에서 보았듯 SQL과 DataFrame API는 같은 옵티마이저를 거칩니다. 성능 차이가 없으니 **가독성으로 고르면** 됩니다.

복잡한 집계·조인은 SQL이, 동적으로 조립하는 파이프라인은 DataFrame API가 읽기 좋은 경우가 많습니다. 둘을 자유롭게 섞을 수 있다는 것이 Spark의 큰 장점입니다.

```
from pyspark.sql import functions as F

orders = spark.createDataFrame([
    (1, "kim", "2026-09-01", 12000),
    (2, "lee", "2026-09-01", 8000),
    (3, "kim", "2026-09-02", 6000),
    (4, "park", "2026-09-03", 15000),
], "oid INT, user STRING, dt STRING, amount INT")
```

8.1 임시 뷰

DataFrame에 SQL에서 쓸 이름을 붙입니다.

```
orders.createOrReplaceTempView("orders")

top = spark.sql("""
    SELECT user, SUM(amount) AS total, COUNT(*) AS n
    FROM orders
    GROUP BY user
    HAVING SUM(amount) > 10000
    ORDER BY total DESC
""")
top.show()
```

출력

user	total	n
kim	18000	2
park	15000	1

- `spark.sql(...)` 은 DataFrame을 반환합니다. 이어서 DataFrame API를 붙여도 됩니다.
- 임시 뷰는 이 **SparkSession 안에서만** 보입니다. 데이터를 복사하지 않고, 계획에 붙인 이름일 뿐입니다.
- `createOrReplaceGlobalTempView` 로 만든 뷰는 같은 애플리케이션의 다른 세션에서도 `global_temp.이름` 으로 보입니다. 쓸 일은 드뭅니다.

8.2 DataFrame을 SQL에 직접 넘기기

뷰 이름을 전역으로 등록하지 않고, 쿼리 문자열에

DataFrame을 바로 끼울 수 있습니다(3.3+).

```
vip = spark.createDataFrame([("kim",)], "user STRING")
spark.sql("""
    SELECT o.* FROM {o} o
    JOIN {v} v ON o.user = v.user
    """, o=orders, v=vip).show()
```

출력

oid	user	dt	amount
3	kim	2026-09-02	6000
1	kim	2026-09-01	12000

8.3 파라미터 바인딩

사용자 입력을 f-string으로 SQL에 끼워 넣으면 SQL 인젝션과 따옴표 버그가 생깁니다. args 로 바인딩하십시오(3.4+).

```
spark.sql(
    "SELECT * FROM orders "
    "WHERE dt >= :since AND amount > :min_amt",
    args={"since": "2026-09-02", "min_amt": 5000},
).show()
```

출력

oid	user	dt	amount
3	kim	2026-09-02	6000
4	park	2026-09-03	15000

{o} 치환은 DataFrame·컬럼 이름 같은 식별자용이고, :이름 은 값용입니다. ? 위치 파라미터와 리스트 args 도

지원합니다.

8.4 섞어 쓰기

```
from pyspark.sql.window import Window

daily = spark.sql("""
    SELECT dt, SUM(amount) AS amt FROM orders GROUP BY dt
""")
# 집계 후 몇 행뿐이라 partitionBy 없이도 무방
w = Window.orderBy("dt").rowsBetween(
    Window.unboundedPreceding, Window.currentRow)
daily.withColumn("cum", F.sum("amt").over(w)) \
    .orderBy("dt").show()
```

출력

dt	amt	cum
2026-09-01	20000	20000
2026-09-02	6000	26000
2026-09-03	15000	41000

반대로 DataFrame 코드 안에서 SQL 식 조각은 `F.expr` ("`...`"), `selectExpr(...)`, `filter("...")` 로 씁니다.

SQL에만 있고 F 에 아직 래퍼가 없는 함수도 `F.expr` 로 부를 수 있습니다.

8.5 CTE와 서브쿼리

```
spark.sql("""
WITH per_user AS (
    SELECT user, SUM(amount) AS total FROM orders
    GROUP BY user
),
ranked AS (
    SELECT *, RANK() OVER (ORDER BY total DESC) AS rk
    FROM per_user
)
SELECT * FROM ranked WHERE rk <= 2
""").show()
```

출력

```
+---+-----+---+
|user|total| rk|
+---+-----+---+
| kim|18000|  1|
| park|15000|  2|
+---+-----+---+
```

CTE는 이름 붙은 서브쿼리일 뿐 **결과를 저장하지 않습니다**. CTE를 두 번 참조하면 두 번 계산될 수 있습니다(옵티마이저가 재사용하기도 하지만 보장되지 않습니다). 값비싼 중간 결과를 여러 번 쓴다면 DataFrame으로 받아 `cache()` 하십시오.

8.6 SQL 파이프 문법 (4.0+)

Spark 4.0은 `|>` 연산자로 위에서 아래로 읽히는 SQL을 지원합니다. DataFrame 체이닝과 사고 순서가 같아 긴

변환에서 읽기 편합니다.

```
spark.sql("""
FROM orders
|> WHERE amount >= 8000
|> AGGREGATE SUM(amount) AS total GROUP BY user
|> ORDER BY total DESC
""").show()
```

출력

```
+-----+-----+
|user|total|
+-----+-----+
|park|15000|
| kim|12000|
| Lee| 8000|
+-----+-----+
```

8.7 카탈로그와 테이블

```
spark.sql("CREATE DATABASE IF NOT EXISTS shop")
orders.write.mode("overwrite").saveAsTable("shop.orders")
for t in spark.catalog.listTables("shop"):
    print(t.name, t.tableType, t.isTemporary)
print(spark.catalog.tableExists("shop.orders"))
spark.sql("DESCRIBE TABLE shop.orders").show()
```

출력

```
orders MANAGED False
orders TEMPORARY True
True
+-----+-----+-----+
|col_name|data_type|comment|
+-----+-----+-----+
|   oid|      int|  NULL|
|  user|   string|  NULL|
|   dt|   string|  NULL|
| amount|      int|  NULL|
+-----+-----+-----+
```

- **관리형(managed) 테이블**: `saveAsTable` 로 경로 없이 만든 테이블. `DROP TABLE` 하면 **데이터 파일도 삭제**됩니다.
- **외부(external) 테이블**: `option("path", ...)` 나 `CREATE TABLE ... LOCATION` 으로 만든 테이블. `DROP` 해도 파일은 남습니다.
- 출력처럼 `listTables` 에는 같은 이름의 **임시 뷰도 함께** 나옵니다. 이름이 겹치면 SQL에서 임시 뷰가 우선하니, 뷰와 테이블 이름을 구분해 짓는 편이 안전합니다.
- 운영 환경에서는 Hive Metastore, AWS Glue, Unity Catalog 같은 공유 카탈로그를 씁니다. 로컬 학습 환경은 세션이 끝나면 사라지는 인메모리 카탈로그입니다.

`spark.table("shop.orders")` 는 `spark.sql("SELECT * FROM shop.orders")` 와 같습니다.

8.8 핵심 정리

- SQL과 DataFrame API는 성능이 같습니다. 섞어 쓰십시오.
- 임시 뷰는 세션 한정 이름표. `{df}` 치환으로 뷰 등록 없이도 쓸 수 있습니다.
- 값은 `args` 로 바인딩. f-string 금지.
- CTE는 캐시가 아닙니다.

UDF와 pandas 연동

9.1 Python UDF: 왜 느린가

내장 함수로 도저히 표현이 안 되는 로직은 Python 함수로 작성해 UDF(User Defined Function)로 등록할 수 있습니다.

```
from pyspark.sql import functions as F
from pyspark.sql import types as T

@F.udf(returnType=T.StringType())
def mask_email(s):
    if s is None:
        return None
    local, _, domain = s.partition("@")
    stars = "*" * (len(local) - 2)
    return local[:2] + stars + "@" + domain

df = spark.createDataFrame(
    [("kimminsu@a.com",), ("lee@b.com",), (None,)],
    "email STRING")
df.select(mask_email("email").alias("masked")).show()
```

출력

```
+-----+
|      masked|
+-----+
|ki*****@a.com|
|      le*@b.com|
|           NULL|
+-----+
```

편리하지만 비용이 큼니다. 1장에서 본 것처럼 Python 함수는 JVM에서 실행할 수 없으므로,

1. Executor(JVM)가 행을 직렬화해 옆의 Python 워커 프로세스로 보내고,
2. Python이 **한 행씩** 함수를 호출하고,
3. 결과를 다시 직렬화해 JVM으로 돌려보냅니다.

게다가 옵티마이저에게 UDF는 **블랙박스**입니다. 필터 푸시다운이나 상수 접기 같은 최적화가 UDF를 통과하지 못합니다.

주의

UDF 작성 시 주의. ① None 입력을 반드시 처리하십시오. ② returnType 과 실제 반환 타입이 다르면 에러 없이 null 이 됩니다(예: IntegerType 인데 float 반환). ③ 함수가 참조하는 전역 객체는 pickle되어 모든 Task로 전송됩니다. 큰 모델·사전을 클로저로 잡으면 매 Task마다 전송 비용이 듭니다.

9.2 pandas UDF: 벡터화된 UDF

pandas UDF는 행 단위 대신 **Arrow 배치 단위**로 데이터를 주고받고, 함수는 `pd.Series` 를 받아 `pd.Series` 를 돌려줍니다. 직렬화가 컬럼 기반이라 싸고, 함수 안에서

pandas/NumPy의 벡터 연산을 쓸 수 있어 Python 호출 횟수가 급감합니다.

```
import pandas as pd

@F.pandas_udf("string")
def mask_email_vec(s: pd.Series) -> pd.Series:
    m = s.str.extract(r"^(.{0,2})(.*)@(.*)$")
    stars = m[1].str.replace(r".", "*", regex=True)
    return m[0] + stars + "@" + m[2]

df.select(mask_email_vec("email").alias("masked")).show()
```

출력

```
+-----+
|      masked      |
+-----+
|ki*****@a.com|
|      le*@b.com  |
|           NULL  |
+-----+
```

함수는 **배치마다** 호출됩니다. 배치에 null만 있거나 행이 0개일 수도 있으니, 컬럼 구조가 입력 내용에 따라 바뀌는 연산 (str.partition 같은)은 피하고 벡터 연산으로 작성하십시오. 타입 힌트가 UDF의 종류를 결정합니다.

시그니처	종류	용도
Series -> Series	스칼라	행 단위 변환 (길이 동일)
Iterator[Series] -> Iterator[Series]	스칼라(반복자)	배치마다 무거운 초기화(모델 로딩)를 한 번만
Series -> 스칼라	집계	groupBy().agg(), 윈도 안에서 사용

```

@F.pandas_udf("double")
def trimmed_mean(v: pd.Series) -> float:
    q1, q9 = v.quantile(0.1), v.quantile(0.9)
    return float(v[(v >= q1) & (v <= q9)].mean())

nums = spark.range(1, 101).selectExpr(
    "id % 2 AS g", "CAST(id AS DOUBLE) AS v")
nums.groupBy("g") \
    .agg(trimmed_mean("v").alias("tm")).show()

```

출력

```

+---+---+
|  g|  tm|
+---+---+
|  0|51.0|
|  1|50.0|
+---+---+

```

반복자 버전은 이렇게 생겼습니다. 머신러닝 모델 추론에 자주 씁니다.

```

from typing import Iterator

@F.pandas_udf("double")
def predict(batches: Iterator[pd.Series]
            ) -> Iterator[pd.Series]:
    model = load_model()          # Task당 한 번만
    for s in batches:
        yield pd.Series(model.predict(s.to_frame()))

```

9.3 얼마나 차이 나는가

같은 계산을 네 가지 방식으로 돌려 보겠습니다(로컬 4코어, 500만 행, 3회 중 최솟값). 첫 실행은 Python 워커 기동

비용이 섞이므로 반복 측정합니다.

```
import time

big = spark.range(5_000_000).select(
    (F.col("id") % 1000).cast("double").alias("x"))

@F.udf("double")
def plus_py(x):
    return x * 2.0 + 1.0

@F.udf("double", useArrow=True)
def plus_arrow(x):
    return x * 2.0 + 1.0

@F.pandas_udf("double")
def plus_pd(x: pd.Series) -> pd.Series:
    return x * 2.0 + 1.0

def bench(label, col, n=3):
    best = float("inf")
    for _ in range(n):
        t = time.perf_counter()
        big.select(F.sum(col)).collect()
        best = min(best, time.perf_counter() - t)
    print(f"{label:<12}{best:6.2f}s")

bench("native", F.col("x") * 2.0 + 1.0)
bench("pandas_udf", plus_pd("x"))
bench("arrow_udf", plus_arrow("x"))
bench("python_udf", plus_py("x"))
```

출력

native	0.21s
pandas_udf	1.14s
arrow_udf	1.42s
python_udf	1.30s

절대 시간은 환경마다 다르지만, 눈여겨볼 것은 **내장 함수와 UDF 사이의 격차**입니다. 어떤 UDF든 JVM↔Python 왕복이 들어가는 순간 몇 배 느려집니다. UDF끼리의 차이는 이

예제처럼 계산이 가벼우면 작고, 함수 본문이 NumPy로 벡터화될 만큼 무거울수록 pandas UDF가 크게 앞섭니다. 결정 규칙은 단순합니다.

1. 내장 함수(`F.*`, SQL 함수, 고차 함수)로 표현할 수 있으면 그것을 쓴다.
2. 안 되면 pandas UDF (벡터화할 수 있는 로직).
3. 행 단위 Python 로직이 꼭 필요하다면 Python UDF. `useArrow=True` (3.5+)로 Arrow 직렬화를 켤 수 있습니다. 이 책의 실행 환경(4.2.0)에서는 `spark.sql.execution.pythonUDF.arrow.enabled` 가 이미 기본 `true` 라 일반 `@F.udf` 도 Arrow로 직렬화되었고, 그래서 위 측정에서 두 UDF의 차이가 작습니다. 3.5 등 이전 버전은 기본값이 `false` 이니 사용하는 버전에서 확인하십시오.

9.4 DataFrame 단위의 pandas 함수

행이나 컬럼이 아니라 **DataFrame 조각 전체**를 pandas로 처리하고 싶을 때가 있습니다.

applyInPandas: 그룹별 pandas 처리

`groupBy(...).applyInPandas(func, schema)` 는 그룹 하나를 통째로 `pd.DataFrame` 으로 넘깁니다. 그룹별 모델 학습, 그룹 내

보간, 복잡한 시계열 처리 등에 씁니다.

```
sales = spark.createDataFrame([
    ("A", 1, 10.0), ("A", 2, None), ("A", 3, 30.0),
    ("B", 1, 5.0), ("B", 2, 7.0), ("B", 3, None),
], "store STRING, day INT, amt DOUBLE")

def fill_gaps(pdf: pd.DataFrame) -> pd.DataFrame:
    pdf = pdf.sort_values("day")
    pdf["amt"] = pdf["amt"].interpolate().ffill()
    return pdf

(sales.groupBy("store")
    .applyInPandas(fill_gaps, schema=sales.schema)
    .orderBy("store", "day").show())
```

출력

store	day	amt
A	1	10.0
A	2	20.0
A	3	30.0
B	1	5.0
B	2	7.0
B	3	7.0

주의

그룹 하나가 **통째로 한 Executor의 메모리**에 올라갑니다. 키 하나에 수천만 행이 물리는 스큐가 있으면 OOM이 납니다. 또 그룹을 모으기 위한 셔플이 일어납니다.

mapInPandas: 파티션별 pandas 처리

셔플 없이 각 파티션을 `pd.DataFrame` 배치 반복자로 넘깁니다.

출력 행 수가 입력과 달라도 됩니다.

```
def only_big(it):
    for pdf in it:
        yield pdf[pdf["amt"] > 6]

sales.mapInPandas(only_big, schema=sales.schema).show()
```

출력

	store	day	amt
	A	1	10.0
	A	3	30.0
	B	2	7.0

두 DataFrame을 같은 키로 묶어 함께 처리하는 `cogroup` (`...`).`applyInPandas` 도 있습니다(`as-of` 조인 등). `Arrow` 배치를 직접 다루는 `mapInArrow` / `applyInArrow` (4.0+)도 있습니다.

9.5 Spark ↔ pandas 변환

```
pdf = sales.filter("store = 'A']").toPandas()
print(type(pdf), pdf.shape)
```

출력

```
<class 'pandas.DataFrame'> (3, 3)
```

`toPandas()` 는 **전체 결과를 Driver로 모읍니다**. 집계나 필터로 충분히 줄인 뒤에만 부르십시오. 반대 방향 `spark.createDataFrame(pdf)` 는 Driver의 데이터를 클러스터로 뿌립니다. 둘 다 `Arrow` 덕분에 빠르지만, Driver 메모리 한계는 그대로입니다.

9.6 pandas API on Spark

pandas 코드를 거의 그대로 분산 실행하고 싶다면 `pyspark.pandas` 가 있습니다(구 Koalas).

```
import pyspark.pandas as ps

psdf = ps.DataFrame({"city": ["Seoul", "Busan", "Seoul"],
                        "amt": [100, 200, 300]})
print(psdf.groupby("city")["amt"].sum().sort_index())
```

출력

city	
Busan	200
Seoul	400

Name: amt, dtype: int64

마이그레이션 첫걸음으로는 유용하지만, 다음 한계를 알고 쓰십시오.

- pandas의 **인덱스와 행 순서** 개념을 흉내 내기 위해 추가 비용(기본 인덱스 생성 등)이 듭니다.
- `iloc`, `shift` 등 순서 의존 연산은 한 파티션으로 몰리는 비싼 연산이 될 수 있습니다.
- 모든 pandas API가 지원되지는 않습니다.

`psdf.to_spark()`, `sdf.pandas_api()` 로 언제든지 오갈 수 있습니다. 장기적으로는 PySpark DataFrame API로 옮기는 것을 권합니다.

9.7 그 밖의 Python 확장 지점

- **Python UDTF (3.5+)**: 행 하나에서 여러 행을 내는 테이블 함수. `@F.udtf` 데코레이터로 클래스를 등록합니다.

- **Python Data Source API (4.0+)**: 사내 API나 특수 포맷을 `spark.read.format("my_source")` 로 읽도록 Python만으로 커넥터를 작성할 수 있습니다.

9.8 핵심 정리

- Python UDF는 행 단위 직렬화 + 옵티마이저 블랙박스. 최후의 수단입니다.
- 우선순위: 내장 함수 → pandas UDF → (Arrow) Python UDF.
- `applyInPandas` 는 그룹 전체가 메모리에. `mapInPandas` 는 셔플 없음.
- `toPandas()` 는 Driver로 전부 가져옵니다. 줄인 뒤에.

성능 튜닝

Spark 튜닝의 원칙은 세 줄로 요약됩니다.

1. **적게 읽는다.** 필요한 컬럼·파티션만. (컬럼 포맷, 파티션 프루닝, 필터 먼저)
2. **적게 옮긴다.** 셔플과 Driver 수집을 줄인다. (브로드캐스트, 사전 집계)
3. **고르게 나눈다.** 모든 Task가 비슷한 양을 처리하게 한다. (파티션 수, 스큐 해소)

설정값을 만지는 것은 그다음입니다. 코드와 데이터 레이아웃이 성능의 대부분을 결정합니다.

```
from pyspark.sql import functions as F
```

10.1 캐싱

2장에서 본 것처럼 액션마다 계획 전체가 재실행됩니다. 같은

중간 결과를 여러 번 쓴다면 캐시하십시오.

```
base = (spark.range(3_000_000)
        .withColumn("k", F.col("id") % 100)
        .withColumn("v", F.rand(seed=1))
        .filter("v > 0.1"))

base.cache()           # 표시만 (게으름)
print(base.count())    # 이때 계산·저장
print(base.storageLevel)

# 이후 액션들은 캐시에서 읽음
print(base.groupBy("k").count().count())
print(round(base.agg(F.avg("v")).first()[0], 4))
base.unpersist()
```

```
출력
2700101
Disk Memory Deserialized 1x Replicated
100
0.5501
```

- `cache()` 는 `persist(MEMORY_AND_DISK)` 의 줄임말입니다 (DataFrame 기준). 메모리에 안 들어가는 파티션은 디스크로 갑니다.
- 저장 수준을 고르려면 `persist(StorageLevel.DISK_ONLY)` 처럼 씁니다(`from pyspark import StorageLevel`).
- `cache()` 자체도 **게으릅니다**. 첫 액션 때 채워집니다. 흔히 `count()` 로 강제로 채웁니다.
- 캐시된 DataFrame은 Spark UI의 **Storage** 탭에서 크기와 메모리/디스크 비율을 확인할 수 있습니다.
- 다 쓰면 `unpersist()` . Executor 메모리는 캐시와 연산이 나눠 씁니다.

캐시하면 좋은 경우: 반복 알고리즘(ML 학습), 한 결과를 여러 출력으로 분기, 탐색적 분석에서 같은 데이터를 계속 조회.

캐시하지 말아야 할 경우: 한 번만 쓰는 데이터. Parquet를 읽고 필터 하나 거는 정도라면 다시 읽는 게 캐시(직렬화 + 메모리 압박)보다 쌀 때가 많습니다.

체크포인트: 계보 끊기

반복문 안에서 DataFrame을 계속 변형하면 계획(lineage)이 끝없이 길어져 분석 시간이 폭증합니다. `checkpoint()` 는 결과를 신뢰할 수 있는 저장소에 쓰고 **계획을 거기서 끊습니다**. `localCheckpoint()` 는 Executor 로컬에 저장하는 빠르지만 덜 안전한 버전입니다.

```
spark.sparkContext.setCheckpointDir("s3://bucket/ckpt")
for i in range(50):
    df = step(df)
    if i % 10 == 0:
        df = df.checkpoint() # 즉시 실행되고 계보 리셋
```

10.2 파티션 수 조절

```
df = spark.range(0, 1_000_000, numPartitions=16)
print(df.rdd.getNumPartitions())
print(df.repartition(4).rdd.getNumPartitions())
print(df.coalesce(4).rdd.getNumPartitions())
print(df.repartition(32, "id").rdd.getNumPartitions())
```

출력

16

4

4

32

	<code>repartition(n, *cols)</code>	<code>coalesce(n)</code>
셔플	있음 (전체 재분배)	없음 (인접 파티션 병합)
방향	늘리기·줄이기 모두	줄이기만
결과 균형	고름	치우칠 수 있음
용도	키 기준 재분배, 병렬성 확대	쓰기 전 파일 수 줄이기

주의

`coalesce(1)` 은 셔플을 피하는 대신 **앞선 연산 전체의 병렬성까지 1로 떨어뜨릴 수 있습니다**(같은 Stage 안에 있으므로). 무거운 계산 뒤에 파일 하나로 쓰고 싶다면 차라리 `repartition(1)` 이 빠를 수 있습니다. 셔플 경계가 생겨 앞 Stage는 병렬로 돌기 때문입니다.

`repartition("k")` 처럼 컬럼을 주면 같은 키가 같은 파티션에 모입니다. 이후 같은 키로 `partitionBy` 해서 쓰면 **파티션 디렉터리당 파일 수가 줄어** 작은 파일 문제를 막습니다.

```
(df.repartition("dt")
  .write.partitionBy("dt").parquet(path))
# dt마다 파일이 (대략) 하나. 파일 크기 상한은
# .option("maxRecordsPerFile", 5_000_000)
```

10.3 AQE: 실행 중 재최적화

Adaptive Query Execution(3.2+ 기본 활성화)은 셔플이 끝날 때마다 **실제 통계**를 보고 남은 계획을 다시 짭니다. 정적인 추정치의 한계를 보완하는 장치로, 세 가지 일을 합니다.

1. **셔플 파티션 병합.** `spark.sql.shuffle.partitions=200` 이어도 실제 데이터가 작으면 작은 파티션들을 합쳐 Task 수를 줄입니다. 목표 크기는 `spark.sql.adaptive.advisoryPartitionSizeInBytes` (기본 64MB).
2. **조인 전략 전환.** 셔플 후 한쪽이 작다고 밝혀지면 Sort Merge를 Broadcast로 바꿉니다(6장에서 확인).
3. **스큐 조인 처리.** 유난히 큰 파티션을 여러 조각으로 쪼개고 상대방을 복제해 조인합니다.

그래서 요즘은 `shuffle.partitions` 를 넉넉히(데이터에 비해 크게) 잡고 AQE가 줄이게 두는 방식이 일반적입니다. `explain()` 에서 `AdaptiveSparkPlan`, `AQEShuffleRead` 가 보이면 AQE가 일하는 중입니다.

10.4 데이터 스큐

증상: Stage의 Task 199개는 몇 초 만에 끝났는데 1개가 수십 분째 돌고 있다. Spark UI의 Task 지표에서 Duration·Shuffle Read Size의 **Max가 Median의 수십 배.**

원인: 특정 키에 데이터가 몰림. `null` 키, `"unknown"`, 기본값, 초대형 고객 등.

진단부터:

```
skewed = spark.range(1_000_000).withColumn(
    "k", F.when(F.col("id") % 10 < 8, F.lit(0))
    .otherwise(F.col("id") % 1000))
(skewed.groupBy("k").count()
    .orderBy(F.desc("count")).show(3))
```

출력

```
+---+-----+
| k| count|
+---+-----+
| 0|800000|
| 28| 1000|
| 29| 1000|
+---+-----+
```

only showing top 3 rows

키 0 에 80%가 몰려 있습니다. 해법은 원인에 따라 고릅니다.

1. **불필요한 키 걸러내기.** `null`·더미 키가 원인이면 조인 전에 빼고 나중에 붙입니다. 가장 싸고 흔한 해결책입니다.
2. **브로드캐스트.** 상대방이 작으면 셔플 자체를 없앱니다.
3. **AQE 스큐 조인.** `spark.sql.adaptive.skewJoin.enabled` (기본 true). 조인에서 자동 처리되지만 **집계와 윈도우에는 적용되지 않습니다.**
4. **솔팅(salting).** 뜨거운 키에 난수 접미사를 붙여 여러 파티션으로 흩은 뒤 두 단계로 집계합니다.

N = 8

```
salted = (skewed
    .withColumn("salt", (F.rand(seed=7) * N).cast("int"))
    # 1단계: (키, 솔트)별 부분 집계
    .groupBy("k", "salt").agg(F.count("*").alias("c"))
    # 2단계: 키별 최종 집계
    .groupBy("k").agg(F.sum("c").alias("count")))
salted.orderBy(F.desc("count")).show(3)
```

출력

```
+---+-----+
| k| count|
+---+-----+
| 0|800000|
| 28| 1000|
| 29| 1000|
+---+-----+
```

only showing top 3 rows

1단계에서 키 0의 행들이 8개 파티션으로 나뉘어 병렬 처리되고, 2단계는 이미 줄어든 부분합만 다룹니다. 사실 `count.sum`은 6장의 부분 집계 덕분에 스큐에 꽤 강합니다. 솔팅이 진짜 필요한 곳은 **부분 집계**가 안 되는 연산(`collect_list`, 정확한 `distinct`, 윈도우, 조인)입니다. 조인 솔팅은 큰 쪽에 `salt`를, 작은 쪽을 `N` 배로 복제(`explode(array(0..N-1))`)한 뒤 (`key`, `salt`)로 조인합니다.

10.5 셔플 줄이기 체크리스트

- **필터와 컬럼 선택을 최대한 앞으로.** 옵티마이저가 대부분 해 주지만, UDF나 캐시 경계를 넘지는 못합니다.
- **조인 전에 집계.** 1억 행을 조인하고 집계하기보다, 먼저 집계해 10만 행으로 줄인 뒤 조인합니다.

- **distinct · orderBy** 는 필요할 때만. 둘 다 전체 셔플입니다. 결과를 보기 좋게 하려는 정렬은 마지막 작은 결과에만.
- 작은 쪽은 브로드캐스트.
- 같은 키로 반복 조인한다면 버킷팅. `bucketBy(n, "key").saveAsTable(...)` 로 저장한 테이블끼리는 같은 키 조인 시 셔플이 생략됩니다. 관리 비용이 있어 반복 조인이 확실할 때만 씁니다.

10.6 작은 파일 문제

파일 수만 개짜리 테이블은 읽을 때마다 파일 목록 조회와 열기 비용이 데이터 처리 시간을 압도합니다. 스트리밍·잡은 `append` ·과도한 `partitionBy` 가 주범입니다.

- 쓰기 전에 `repartition / coalesce` 로 파일 수를 조절합니다. 목표는 파일당 **128MB-1GB**.
- 읽는 쪽에서는 `spark.sql.files.maxPartitionBytes` 가 작은 파일들을 한 Task로 묶어 줍니다(파일 열기 비용은 남음).
- Delta/Iceberg는 `OPTIMIZE · rewrite_data_files` 로 주기적 컴팩션을 지원합니다.

10.7 메모리와 스펴

Executor 메모리는 대략 이렇게 나뉩니다.

- **실행(execution) 메모리**: 셔플·조인·정렬·집계의 작업 공간.
- **저장(storage) 메모리**: 캐시, 브로드캐스트 변수. 실행 메모리와 경계가 유동적입니다(unified memory).

- **오버헤드(overhead):** JVM 힙 밖. **Python** 워커와 Arrow 버퍼가 여기를 씁니다. pandas UDF를 많이 쓰면 `spark.executor.memoryOverhead` (또는 `spark.executor.pyspark.memory`)를 늘려야 할 수 있습니다. 컨테이너가 이유 없이 죽는다면(YARN/K8s의 OOMKilled) 대개 이쪽입니다.

실행 메모리가 부족하면 Spark는 중간 데이터를 디스크에 **스필(spill)** 하고 계속 진행합니다. UI의 `Spill (Memory/Disk)`가 크면 파티션당 데이터가 너무 많다는 신호입니다. 파티션 수를 늘리십시오.

10.8 자주 만지는 설정

설정	기본	의미
<code>spark.sql.shuffle.partitions</code>	200	셔플 후 파티션 수 (AQE가 병합)
<code>spark.sql.adaptive.enabled</code>	true	AQE
<code>spark.sql.autoBroadcastJoinThreshold</code>	10MB	자동 브로드캐스트 상한 (-1 = 끄)
<code>spark.sql.files.maxPartitionBytes</code>	128MB	파일 읽기 파티션 크기
<code>spark.executor.memory / cores</code>	-	Executor 힙 / 코어
<code>spark.executor.memoryOverhead</code>	힙의 10%	힙 밖 메모리 (Python 포함)
<code>spark.driver.memory</code>	1g	Driver 힙 (collect·broadcast)

설정	기본	의미
<code>spark.driver.maxResultSize</code>	1g	Driver로 모을 결과 상한
<code>spark.sql.sources.partitionOverwriteMode</code>	static	dynamic : 해당 파티션만 덮어쓰기

`spark.sql.*` 설정 대부분은 실행 중 `spark.conf.set(...)` 으로 바꿀 수 있지만, 메모리·코어처럼 프로세스 기동 시 정해지는 값은 세션을 만들 때(또는 `spark-submit` 에서) 줘야 합니다.

10.9 안티패턴 모음

- 중간 확인용 `count()` / `show()` 를 파이프라인 곳곳에 남겨 두기 → 매번 전체 재실행.
- 큰 DataFrame에 `collect()` , `toPandas()` → Driver OOM.
- `for` 루프로 `withColumn` 수십 번 → 계획 폭발. `select / withColumns` 로.
- `for` 루프로 필터 조건마다 따로 쿼리 → 한 번의 `groupBy` 나 `when` 으로.
- 내장 함수로 되는 일을 UDF로.
- 고카디널리티 컬럼으로 `partitionBy` → 작은 파일 수백만 개.
- 스키마 추론(`inferSchema`)에 기대기 → 이중 스캔, 타입 불안정.

10.10 핵심 정리

- 적게 읽고, 적게 옮기고, 고르게 나누기.
- 캐시는 여러 번 쓰는 결과에만, 다 쓰면 `unpersist`.
- `repartition` 은 셔플, `coalesce` 는 병합. 파일 수 조절에 씁니다.
- AQE가 파티션 병합·조인 전환·스큐 조인을 자동 처리합니다.
- 스큐는 진단(키 분포) → 걸러내기/브로드캐스트/솔팅.

RDD: 알아야 할 저수준 API

RDD(Resilient Distributed Dataset)는 Spark 초기의 핵심 추상화입니다. DataFrame은 RDD 위에 스키마와 옵티마이저를 얹은 것이고, 지금도 모든 DataFrame 연산은 결국 RDD 연산으로 실행됩니다.

새 코드에서 RDD를 쓸 이유는 거의 없습니다. RDD 연산은 옵티마이저의 도움을 받지 못하고, PySpark에서는 모든 연산이 Python 람다라 UDF처럼 직렬화 비용을 치릅니다. Spark Connect에서는 아예 쓸 수 없습니다. 그래도 옛 코드를 읽고, 가끔 필요한 저수준 조작을 하려면 개념은 알아야 합니다.

11.1 RDD라는 이름의 뜻

- **Resilient**: 각 RDD는 자신이 어떤 부모에서 어떤 연산으로 만들어졌는지(**계보, lineage**)를 기억합니다. Executor가 죽어 파티션을 잃으면, 데이터를 복제해 두는 대신 계보를 따라 **그 파티션만 다시 계산**합니다. Spark 장애 복구의 기본 원리이고, DataFrame에도 그대로 적용됩니다.
- **Distributed**: 파티션으로 나뉘어 클러스터에 흩어져 있습니다.
- **Dataset**: 임의의 Python 객체의 모음입니다. 스키마가 없습니다.

11.2 맛보기

```
sc = spark.sparkContext
lines = sc.parallelize([
    "spark is fast", "pandas is handy",
    "spark is lazy",
], numSlices=2)

counts = (lines
    .flatMap(lambda s: s.split())
    .map(lambda w: (w, 1))
    .reduceByKey(lambda a, b: a + b)
    .sortBy(lambda kv: -kv[1]))
print(counts.take(4))
```

출력

```
[('is', 3), ('spark', 2), ('fast', 1), ('handy', 1)]
```

전통의 워드카운트입니다. 같은 일을 DataFrame으로 하면
이렇습니다. 옵티마이저와 코드 생성의 혜택을 받고, Python
람다가 없습니다.

```
from pyspark.sql import functions as F
texts = ["spark is fast", "pandas is handy",
         "spark is lazy"]
df = spark.createDataFrame([(t,) for t in texts],
                           ["line STRING"])
(df.select(F.explode(F.split("line", " ")).alias("w"))
 .groupBy("w").count()
 .orderBy(F.desc("count"), "w").show(4))
```

출력

w	count
is	3
spark	2
fast	1
handy	1

only showing top 4 rows

11.3 기억해 둘 RDD 개념

개념	설명
map, flatMap, filter	좁은 변환
reduceByKey	키별 결합. 셔플 전 부분 결합 (combiner) 수행
groupByKey	키별로 모든 값을 모음. 부분 결합 없음 → 느리고 OOM 위험 . reduceByKey / aggregateByKey 를 쓸 것

개념	설명
mapPartitions	파티션 단위 함수. 커넥션 같은 비싼 자원을 파티션당 한 번만 여는 패턴
broadcast(v)	읽기 전용 값을 Executor마다 한 번만 전송
accumulator	Executor → Driver 방향의 누적 카운터. 디버깅·지표용

11.4 브로드캐스트 변수와 어큐물레이터

이 둘은 DataFrame 시대에도 가끔 씁니다.

```
lookup = sc.broadcast({"kim": "gold", "lee": "basic"})
bad = sc.accumulator(0)

def tier(name):
    t = lookup.value.get(name)
    if t is None:
        bad.add(1)
    return t

print(sc.parallelize(["kim", "lee", "park"])
      .map(tier).collect())
print("unknown:", bad.value)
```

```
출력
['gold', 'basic', None]
unknown: 1
```

- 큰 사전·모델을 UDF 안에서 쓸 때 클로저로 잡는 대신 `sc.broadcast` 로 감싸면 Task마다가 아니라 **Executor마다 한 번** 전송됩니다.
- 어큐물레이터는 **액션 안에서만 정확합니다**. 변환(`map`) 안에서 더한 값은 Task 재시도나 재계산 때 중복 집계될 수

있습니다. 비즈니스 수치가 아니라 대략적인 모니터링 용도로만 쓰십시오.

11.5 DataFrame ↔ RDD

```
rdd = df.rdd                # Row 객체의 RDD
df2 = rdd.toDF(schema)
# 또는 spark.createDataFrame(rdd, schema)
```

`df.rdd` 로 넘어가는 순간 옵티마이저의 세계를 벗어나고, 모든 행이 Python 객체로 직렬화됩니다. `df.rdd.getNumPartitions()` 처럼 **메타 정보를 볼 때만** 쓰십시오.

11.6 핵심 정리

- RDD = 계보를 기억하는 분산 컬렉션. 장애 복구의 원리입니다.
- 새 코드는 DataFrame. RDD는 읽을 줄만 알면 됩니다.
- `groupByKey` 대신 `reduceByKey`. 브로드캐스트는 Executor당 1회 전송.

Structured Streaming

12.1 핵심 아이디어: 끝없이 늘어나는 테이블

Structured Streaming은 스트림을 **계속 행이 추가되는 무한한 테이블**로 봅니다. 여러분은 이 테이블에 대해 평소처럼 DataFrame 쿼리를 쓰고, Spark는 새 데이터가 들어올 때마다 그 쿼리를 **중분으로** 실행해 결과를 갱신합니다.

그래서 배치 코드와 스트리밍 코드가 거의 같습니다. `spark.read` → `spark.readStream`, `df.write` → `df.writeStream`으로 바뀌는 정도입니다.

기본 실행 방식은 **마이크로 배치**입니다. 트리거 간격마다 새로 도착한 데이터를 작은 배치로 처리합니다. 지연 시간은 보통 수백 ms ~ 수 초입니다.

12.2 예제: 폴더에 쌓이는 JSON 이벤트

```
import os, json, shutil
from pyspark.sql import functions as F

src = "/tmp/book_stream/in"
ckpt = "/tmp/book_stream/ckpt"
shutil.rmtree("/tmp/book_stream", ignore_errors=True)
os.makedirs(src)

def drop_file(name, events):
    with open(f"{src}/{name}.json", "w") as f:
        for e in events:
            f.write(json.dumps(e) + "\n")

drop_file("b1", [
    {"user": "kim", "ts": "2026-09-21 10:00:05",
     "amt": 10},
    {"user": "lee", "ts": "2026-09-21 10:00:40",
     "amt": 20},
    {"user": "kim", "ts": "2026-09-21 10:01:10",
     "amt": 5},
])
```

스트리밍 소스는 **스키마를 반드시 명시**해야 합니다. 도중에 스키마가 바뀌면 곤란하기 때문입니다.

```
events = (spark.readStream
    .schema("user STRING, ts TIMESTAMP, amt INT")
    .option("maxFilesPerTrigger", 1)
    .json(src))
print(events.isStreaming)

per_min = (events
    .withWatermark("ts", "2 minutes")
    .groupBy(F.window("ts", "1 minute"), "user")
    .agg(F.sum("amt").alias("amt")))
```

출력
True

이제 쿼리를 시작합니다. 결과는 `foreachBatch` 싱크로 받겠습니다. 마이크로 배치마다 결과를 **평범한 배치 DataFrame**으로 넘겨받아 원하는 대로 쓸 수 있는 만능 싱크입니다. 여기서는 배치 번호를 붙여 Parquet에 추가합니다. `availableNow` 트리거는 **지금까지 쌓인 데이터만 처리하고 멈춥니다.**

```
out = "/tmp/book_stream/out"

def sink(batch_df, batch_id):
    (batch_df.withColumn("batch", F.lit(batch_id))
     .write.mode("append").parquet(out))

def run_once():
    q = (per_min.writeStream
         .foreachBatch(sink)
         .outputMode("update")
         .option("checkpointLocation", ckpt)
         .trigger(availableNow=True)
         .start())
    q.awaitTermination()
    return q

def show_out():
    (spark.read.parquet(out)
     .select("batch", "window.start", "user", "amt")
     .orderBy("batch", "start", "user").show())

run_once()
show_out()
```

출력

batch	start	user	amt
0	2026-09-21 10:00:00	kim	10
0	2026-09-21 10:00:00	lee	20
0	2026-09-21 10:01:00	kim	5

새 파일이 도착한 뒤 다시 실행하면, **체크포인트** 덕분에 이미 처리한 파일은 건너뛰고 새 파일만 처리합니다. 집계 상태 (state)도 체크포인트에서 이어받으므로 10:01 구간 kim의 합계가 5에서 12로 누적됩니다.

```
drop_file("b2", [
  {"user": "kim", "ts": "2026-09-21 10:01:30",
   "amt": 7},
  {"user": "lee", "ts": "2026-09-21 10:02:00",
   "amt": 1},
])
q = run_once()
show_out()
```

출력

batch	start	user	amt
0	2026-09-21 10:00:00	kim	10
0	2026-09-21 10:00:00	lee	20
0	2026-09-21 10:01:00	kim	5
2	2026-09-21 10:01:00	kim	12
2	2026-09-21 10:02:00	lee	1

update 모드라 두 번째 배치는 **바뀐 행만** 내보냈습니다(10:00 구간은 그대로라 다시 나오지 않음). 배치 번호가 1을 건너뛴 것은 워터마크가 전진한 뒤 데이터 없이 상태만 정리하는 **no-data 배치**가 끼어들었기 때문입니다. 실제 운영에서는 이런 갱신 행을 foreachBatch 안에서 MERGE 로 대상 테이블에 upsert합니다(아래 참고).

12.3 출력 모드

모드	매 트리거마다 내보내는 것	쓰임
append	새로 확정된 행만	집계 없는 변환, 워터마크가 있는 집계
update	변경된 행만	집계 결과를 upsert하는 싱크
complete	결과 테이블 전체	작은 집계, 대시보드

12.4 이벤트 시간과 워터마크

스트림 데이터는 늦게 도착합니다. 10:00의 이벤트가 네트워크 문제로 10:05에 들어올 수 있지요. Spark는 **이벤트 시간**(`ts`) 기준으로 윈도우를 계산하고, 늦게 온 데이터도 해당 윈도우에 반영합니다.

문제는 “언제까지 기다리느냐”입니다. 무한히 기다리면 모든 윈도우의 상태를 영원히 들고 있어야 합니다. **워터마크**가 그 한도입니다.

```
.withWatermark("ts", "2 minutes")
```

“지금까지 본 가장 늦은 이벤트 시각 - 2분”보다 오래된 윈도우는 **닫힌 것으로** 간주합니다. 닫힌 윈도우의 상태는 메모리에서 지우고, 그보다 늦게 온 데이터는 버립니다. `append` 모드에서는 윈도우가 닫혀야 결과가 한 번 출력됩니다.

워터마크 없이 스트리밍 집계를 돌리면 상태가 끝없이 커져 결국 메모리가 터집니다. **스트리밍 집계·조인·중복 제거**에는

워터마크를 거십시오. 중복 제거는 `dropDuplicatesWithinWatermark (3.5+)`가 편합니다.

12.5 트리거

```
# 30초마다 마이크로 배치
.trigger(processingTime="30 seconds")
# 쌓인 것만 처리하고 종료
.trigger(availableNow=True)
# 기본(생략): 이전 배치가 끝나는 즉시 다음 배치
```

`availableNow` 는 스트리밍 코드를 **증분 배치 작업**처럼 쓰게 해 줍니다. 스케줄러로 한 시간마다 돌려도, 체크포인트가 “어디까지 처리했는지”를 기억하므로 정확히 새 데이터만 처리합니다. 상시 클러스터 없이 스트리밍의 이점을 누리는 실용적인 패턴입니다.

12.6 체크포인트와 정확히 한 번

`checkpointLocation` 에는 처리한 소스 오프셋, 집계 상태, 커밋 로그가 저장됩니다. 쿼리가 죽었다가 재시작되면 여기서 이어 갑니다. 재생 가능한 소스(Kafka, 파일)와 멍등 싱크(파일, Delta)를 조합하면 **exactly-once**가 보장됩니다.

주의

체크포인트는 쿼리의 **정체성**입니다. 쿼리 로직(특히 집계 키나 상태 구조)을 크게 바꾸면 기존 체크포인트와 호환되지 않을 수 있습니다. 운영 중인 스트림을 바꿀 때는 새 체크포인트 경로로 시작하는 계획을 세우십시오. 체크포인트는 반드시 영속 저장소(S3, HDFS)에 두십시오.

12.7 Kafka와 foreachBatch

실무에서 가장 흔한 소스는 Kafka입니다(별도 패키지 필요).

```
raw = (spark.readStream.format("kafka")
      .option("kafka.bootstrap.servers", "broker:9092")
      .option("subscribe", "orders")
      .option("startingOffsets", "latest")
      .load())
orders = raw.select(
    F.from_json(F.col("value").cast("string"),
                order_schema).alias("o")).select("o.*")
```

앞의 예제에서 본 `foreachBatch` 로 Delta 테이블에 upsert하는 전형적인 모양은 이렇습니다.

```
def upsert(batch_df, batch_id):
    batch_df.createOrReplaceTempView("updates")
    batch_df.sparkSession.sql("""
        MERGE INTO target t USING updates u
        ON t.id = u.id
        WHEN MATCHED THEN UPDATE SET *
        WHEN NOT MATCHED THEN INSERT *
    """)

orders.writeStream.foreachBatch(upsert) \
    .option("checkpointLocation", ckpt).start()
```

12.8 쿼리 관리

```
q = df.writeStream...start()
q.status           # 현재 상태
q.lastProgress     # 직전 배치의 처리량·지연·상태 크기
q.stop()
spark.streams.active # 실행 중인 쿼리 목록
q.awaitTermination() # 스크립트가 바로 끝나지 않게
```

12.9 핵심 정리

- 스트림 = 무한히 늘어나는 테이블. 배치 코드와 거의 같습니다.
- 스키마 명시, 체크포인트 필수(영속 저장소).
- 이벤트 시간 집계에는 워터마크. 없으면 상태가 무한히 자랍니다.
- `availableNow` 트리거 = 증분 배치. `foreachBatch` = 만능 싱크.

배포, 테스트, 디버깅

13.1 애플리케이션 구조

노트북에서 짠 코드를 운영에 올릴 때는 다음 모양을 권합니다.

```
# jobs/daily_sales.py
import argparse
from pyspark.sql import SparkSession, DataFrame
from pyspark.sql import functions as F

def transform(orders: DataFrame, users: DataFrame
              ) -> DataFrame:
    """순수 변환: I/O 없음 → 단위 테스트 가능"""
    return (orders.join(F.broadcast(users), "user")
            .groupBy("dt", "tier")
            .agg(F.sum("amount").alias("amt")))

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--dt", required=True)
    args = p.parse_args()

    spark = (SparkSession.builder
             .appName(f"daily_sales:{args.dt}")
             .getOrCreate())
    orders = spark.read.table("raw.orders") \
        .where(F.col("dt") == args.dt)
    users = spark.read.table("dim.users")
    (transform(orders, users)
     .write.mode("overwrite")
     .option("partitionOverwriteMode", "dynamic")
     .insertInto("mart.daily_sales"))
    spark.stop()

if __name__ == "__main__":
    main()
```

핵심은 **I/O와 변환의 분리**입니다. `transform` 은 `DataFrame`을 받아 `DataFrame`을 돌려주는 순수 함수이므로 작은 입력으로 테스트할 수 있습니다. 여러 변환을 이어 붙일 때는 `df.transform(f)` 체이닝이 깔끔합니다.

```
result = (raw.transform(clean)
          .transform(enrich, users) # 3.3+ 인자 전달
          .transform(aggregate))
```

13.2 spark-submit

```
spark-submit \
  --master yarn --deploy-mode cluster \
  --name daily_sales \
  --num-executors 20 \
  --executor-cores 4 --executor-memory 12g \
  --driver-memory 4g \
  --conf spark.sql.shuffle.partitions=800 \
  --conf spark.dynamicAllocation.enabled=true \
  --py-files deps.zip \
  --archives pyenv.tar.gz#environment \
  jobs/daily_sales.py --dt 2026-09-21
```

- **deploy-mode:** `client` 는 Driver가 submit한 머신에서, `cluster` 는 클러스터 안에서 돌립니다. 운영 배치는 `cluster` 가 일반적입니다.
- **Executor 크기:** 코어 4-5개, 메모리는 코어당 수 GB가 무난한 출발점입니다. 코어가 너무 많은 거대 Executor는 GC와 HDFS I/O 처리량 문제가 생깁니다.
- **동적 할당**(`dynamicAllocation`): 작업량에 따라 Executor 수를 늘리고 줄입니다. 공유 클러스터에서 권장됩니다.
- **Python 의존성:** 순수 Python 모듈은 `--py-files` (zip), 네이티브 확장이 있는 패키지(pandas, numpy 등)는 `ven`

v-pack / conda-pack 으로 환경 전체를 압축해 --archives 로 배포하고 PYSPARK_PYTHON=./environment/bin/python 을 지정합니다. Kubernetes라면 의존성을 담은 컨테이너 이미지가 가장 깔끔합니다.

- **JVM 패키지**(Kafka 커넥터, Delta 등): --packages groupId:artifactId:version .

Databricks, EMR, Dataproc 같은 관리형 플랫폼은 이 설정의 상당 부분을 대신 해 주지만, 개념은 같습니다.

13.3 테스트

pytest 픽스처로 로컬 세션 하나를 테스트 세션 전체에서 공유합니다. 세션 생성은 수 초가 걸리므로 테스트마다 만들지 마십시오.

```
# conftest.py
import pytest
from pyspark.sql import SparkSession

@pytest.fixture(scope="session")
def spark():
    s = (SparkSession.builder.master("local[2]")
        .config("spark.sql.shuffle.partitions", "2")
        .config("spark.ui.enabled", "false")
        .getOrCreate())
    yield s
    s.stop()
```

shuffle.partitions 를 작게 잡는 것만으로 테스트 속도가 크게 달라집니다. 비교에는 pyspark.testing 의 도우미(3.5+)를 씁니다.

```
from pyspark.testing import (assertDataFrameEqual,
                             assertSchemaEqual)
from pyspark.sql import functions as F

def transform(orders, users):
    return (orders.join(users, "user")
            .groupBy("tier")
            .agg(F.sum("amount").alias("amt")))

orders = spark.createDataFrame(
    [("kim", 10), ("lee", 5), ("kim", 7)],
    "user STRING, amount INT")
users = spark.createDataFrame(
    [("kim", "gold"), ("lee", "basic")],
    "user STRING, tier STRING")
expected = spark.createDataFrame(
    [("gold", 17), ("basic", 5)],
    "tier STRING, amt BIGINT")

actual = transform(orders, users)
assertDataFrameEqual(actual, expected) # 행 순서 무시
assertSchemaEqual(actual.schema, expected.schema)
print("ok")
```

출력
ok

assertDataFrameEqual 은 기본적으로 행 순서를 무시하고, 실패하면 어떤 행이 다른지 diff를 보여 줍니다. sum(INT) 의 결과가 BIGINT 라는 것처럼, 스키마 테스트가 예상치 못한 타입 변화를 잡아 주는 경우가 많습니다.

13.4 Spark UI 읽는 법

느린 작업을 조사하는 순서입니다(로컬은 `localhost:4040`, 끝난 작업은 History Server).

1. **SQL / DataFrame 탭**: 쿼리별 실행 계획이 그래프로 나오고, 각 노드에 **실제 행 수·시간·셔플 크기**가 찍힙니다. `explain()`의 실측 버전입니다. 어디서 행 수가 폭발했는지 (조인 키 중복), 어떤 조인 전략이 쓰였는지 여기서 확인합니다.
2. **Stages 탭**: 가장 오래 걸린 Stage를 엽니다. **Summary Metrics** 표에서 Duration·Shuffle Read의 Min/Median/Max를 봅니다. Max가 Median의 몇 배 이상이면 **스큐**입니다.
3. 같은 표에서 **Spill**이 크면 파티션당 데이터가 너무 많습니다 → 파티션 수 늘리기.
4. **GC Time**이 Duration의 10%를 넘으면 메모리 압박입니다.
5. **Executors 탭**: 실패한 Task, 죽은 Executor, 캐시 사용량.
6. **Storage 탭**: 캐시된 데이터의 크기와 메모리/디스크 비율.

13.5 에러 메시지 읽기

PySpark 에러는 Python 트레이스백 안에 JVM 스택트레이스가 들어 있어 길고 무섭습니다. 요령은 둘입니다.

- **에러 클래스를 찾으십시오.** Spark 3.4+의 에러는 `[UNRESOLVED_COLUMN.WITH_SUGGESTION]` 처럼 대괄호 안에 분류가

있습니다. 대부분 메시지에 해결책(“Did you mean…”)까지 들어 있습니다.

- **Python 워커 에러는 `PythonException` 으로 감싸져**
옵니다. 그 안에서 여러분 UDF의 트레이스백을
찾으십시오.

```
import textwrap
from pyspark.errors import AnalysisException
try:
    orders.select("usr")
except AnalysisException as e:
    print(e.getCondition())
    print(textwrap.fill(str(e).splitlines()[0], 56))
```

출력
UNRESOLVED_COLUMN.WITH_SUGGESTION
[UNRESOLVED_COLUMN.WITH_SUGGESTION] A column, variable,
or function parameter with name `usr` cannot be
resolved. Did you mean one of the following? [`user`,
`amount`]. SQLSTATE: 42703;

13.6 자주 만나는 문제

증상	흔한 원인	대응
Driver OutOfMemoryError	collect / toPandas , 큰 브로드캐스트, 너무 많은 파티션 메타데이터	결과 줄이기, 브로드캐스트 크기 확인, driver.memory
Container killed ... exceeding memory limits / OOMKilled	힙 밖 메모리(Python 워커, Arrow) 부족	executor.memoryOver head 증가, 배치 크기 (arrow.maxRecordsPer Batch) 감소
한 Task만 끝나지 않음	데이터 스큐	10장: 키 분포 확인, 솔팅/브로드캐스트

증상	흔한 원인	대응
UNRESOLVED_COLUMN	오타, 조인 후 모호한 이름	제안된 이름 확인, alias 로 한정
CAST_INVALID_INPUT , DIVIDE_BY_ZERO	4.x ANSI 모드	try_cast , try_divide
PicklingError / SparkContext 직렬화 에러	UDF 안에서 spark·DataFrame 참조	UDF에는 순수 Python 값만. DataFrame은 조인으로
브로드캐스트 타임아웃	브로드캐스트 대상이 큼·느림	힌트 제거, broadcastTimeout 증가
작업이 점점 느려짐 (반복문)	계보(lineage) 증가	checkpoint / localCh eckpoint
결과 행 수가 너무 많음	조인 키 중복	키 고유성 확인 후 조인
결과 행 수가 너무 적음	null 비교, inner 조인에서 키 누락	eqNullSafe , left 조인으로 비교

13.7 핵심 정리

- 변환은 순수 함수로 분리하고 df.transform 으로 잇기 → 테스트 가능.
- 테스트는 세션 픽스처 공유 + 작은 shuffle.partitions + assertDataFrameEqual .
- 느리면 UI: SQL 탭(실측 계획) → Stage 요약 지표(스큐·스필·GC).
- 에러는 대괄호 속 에러 클래스부터 읽습니다.

부록 A. pandas → PySpark 번역표

pdf 는 pandas DataFrame, df 는 Spark DataFrame입니다. F 는 pyspark.sql.functions , W 는 Window 입니다.

만들기·보기

pandas	PySpark
pd.read_csv(p)	spark.read.csv(p, header=True, schema=...)
pd.read_parquet(p)	spark.read.parquet(p)
pdf.to_parquet(p)	df.write.parquet(p) (디렉터리)
pd.DataFrame(data)	spark.createDataFrame(data, schema)
pdf.head(5)	df.show(5) / df.limit(5) / df.take(5)
len(pdf)	df.count() (전체 스캔)
pdf.shape	(df.count(), len(df.columns))
pdf.dtypes / pdf.info()	df.dtypes / df.printSchema()
pdf.describe()	df.describe() / df.summary()
pdf.columns	df.columns

선택·변환

pandas	PySpark
pdf[["a", "b"]]	df.select("a", "b")

pandas	PySpark
<code>pdf["c"] = pdf.a + 1</code>	<code>df = df.withColumn("c", F.col("a") + 1)</code>
<code>pdf.assign(c=..., d=...)</code>	<code>df.withColumns({"c": ..., "d": ...})</code>
<code>pdf.rename(columns={...})</code>	<code>df.withColumnsRenamed({...})</code>
<code>pdf.drop(columns=["a"])</code>	<code>df.drop("a")</code>
<code>pdf.a.astype("int")</code>	<code>F.col("a").cast("int") / .try_cast(...)</code>
<code>np.where(c, x, y)</code>	<code>F.when(c, x).otherwise(y)</code>
<code>pdf.a.map(dict)</code>	<code>F.create_map(...)[F.col("a")]</code> 또는 <code>조인</code>
<code>pdf.a.apply(f)</code>	내장 함수 → <code>pandas_udf</code> → <code>udf</code> 순
<code>pdf.a.str.lower()</code>	<code>F.lower("a")</code>
<code>pdf.a.str.contains("x")</code>	<code>F.col("a").contains("x") / .rlike(re)</code>
<code>pdf.a.str.split("-")</code>	<code>F.split("a", "-")</code>
<code>pd.to_datetime(pdf.a)</code>	<code>F.to_timestamp("a", fmt)</code>
<code>pdf.a.dt.year</code>	<code>F.year("a")</code>
<code>pdf.a.clip(0, 10)</code>	<code>F.least(F.greatest("a", F.lit(0)), F.lit(10))</code>
<code>pdf.a.round(2)</code>	<code>F.round("a", 2)</code>

필터·정렬·중복

pandas	PySpark
<code>pdf[pdf.a > 1]</code>	<code>df.filter(F.col("a") > 1)</code>
<code>pdf.query("a > 1")</code>	<code>df.filter("a > 1")</code>
<code>pdf[pdf.a.isin([1, 2])]</code>	<code>df.filter(F.col("a").isin(1, 2))</code>
<code>pdf[pdf.a.between(1, 5)]</code>	<code>df.filter(F.col("a").between(1, 5))</code>
<code>pdf.sort_values("a", ascending=False)</code>	<code>df.orderBy(F.desc("a"))</code>
<code>pdf.nlargest(3, "a")</code>	<code>df.orderBy(F.desc("a")).limit(3)</code>
<code>pdf.drop_duplicates()</code>	<code>df.distinct()</code>
<code>pdf.drop_duplicates(["a"])</code>	<code>df.dropDuplicates(["a"])</code> (남는 행 임의)
<code>pdf.sample(frac=0.1)</code>	<code>df.sample(fraction=0.1, seed=1)</code>

결측값

pandas	PySpark
<code>pdf.isna() / notna()</code>	<code>F.col("a").isNull() / .isNotNull()</code>
<code>pdf.fillna(0)</code>	<code>df.na.fill(0) / df.fillna(0)</code>
<code>pdf.fillna({"a": 0})</code>	<code>df.na.fill({"a": 0})</code>
<code>pdf.dropna(subset=["a"])</code>	<code>df.na.drop(subset=["a"])</code>

pandas	PySpark
<code>pdf.a.ffill()</code>	<code>F.last("a", ignorenulls=True).over(w.rowsBe tween(W.unboundedPreceding, 0))</code>
<code>pdf.a.isna().sum()</code>	<code>df.select(F.count_if(F.col("a") .isNull()))</code>

집계

pandas	PySpark
<code>pdf.groupby("k").a.sum()</code>	<code>df.groupBy("k").agg(F.sum("a"))</code>
<code>pdf.groupby("k").agg(s=("a", "sum"))</code>	<code>df.groupBy("k").agg(F.sum("a"). alias("s"))</code>
<code>pdf.groupby("k").size()</code>	<code>df.groupBy("k").count()</code>
<code>pdf.a.value_counts()</code>	<code>df.groupBy("a").count().orderBy (F.desc("count"))</code>
<code>pdf.a.nunique()</code>	<code>F.countDistinct("a") / F.approx_count_distinct("a")</code>
<code>pdf.groupby("k").a.apply(list)</code>	<code>F.collect_list("a")</code>
<code>pdf.pivot_table(index, columns, values, aggfunc)</code>	<code>df.groupBy(i).pivot(c, [...]).agg(...)</code>
<code>pdf.melt(id_vars, value_vars)</code>	<code>df.unpivot(ids, values, "var", "val") (3.4+)</code>
<code>pdf.a.quantile(0.5)</code>	<code>F.percentile_approx("a", 0.5) / F.median("a")</code>
<code>pdf.groupby("k").apply(f)</code>	<code>df.groupBy("k").applyInPandas(f , schema)</code>

윈도(transform·shift·rank·rolling)

w = W.partitionBy("k").orderBy("t")

pandas	PySpark
groupby("k").a.transform("sum")	F.sum("a").over(W.partitionBy("k"))
groupby("k").a.shift(1)	F.lag("a", 1).over(w)
groupby("k").a.shift(-1)	F.lead("a", 1).over(w)
groupby("k").a.cumsum()	F.sum("a").over(w.rowsBetween(W.unboundedPreceding, 0))
groupby("k").a.rank(method="first")	F.row_number().over(w)
groupby("k").a.rank(method="min")	F.rank().over(w)
groupby("k").a.rank(method="dense")	F.dense_rank().over(w)
groupby("k").a.rolling(3).mean()	F.avg("a").over(w.rowsBetween(-2, 0))
groupby("k").a.diff()	F.col("a") - F.lag("a").over(w)
groupby("k").cumcount()	F.row_number().over(w) - 1
pdf.resample("1h").sum()	df.groupBy(F.window("ts", "1 hour")).agg(...)

결합

pandas	PySpark
pdf.merge(o, on="k")	df.join(o, "k")

pandas	PySpark
<code>pdf.merge(o, on="k", how="left")</code>	<code>df.join(o, "k", "left")</code>
<code>pdf.merge(o, left_on="a", right_on="b")</code>	<code>df.join(o, df.a == o.b)</code>
<code>pdf[pdf.k.isin(o.k)]</code>	<code>df.join(o, "k", "left_semi")</code>
<code>pdf[~pdf.k.isin(o.k)]</code>	<code>df.join(o, "k", "left_anti")</code>
<code>pd.concat([a, b])</code>	<code>a.unionByName(b)</code>
<code>pd.merge_asof(...)</code>	원도 + 조인 또는 <code>cogroup().applyInPandas</code>

그 밖의 차이

- **인덱스가 없습니다.** `reset_index`, `loc`, `iloc`, `set_index`에 대응하는 개념이 없습니다. 모든 것은 컬럼입니다.
- **행 순서가 없습니다.** `orderBy`로 명시하지 않은 순서는 보장되지 않습니다. `head()`의 결과도 실행마다 다를 수 있습니다.
- **반복하지 마십시오.** `iterrows()`에 해당하는 `df.toLocalIterator()`가 있지만, 행 단위 Python 반복은 분산의 이점을 모두 버립니다.
- **변경이 아니라 재할당.** `df = df.withColumn(...)`처럼 결과를 받아야 합니다.

부록 B. 메서드 치트 시트

전제: `from pyspark.sql import functions as F, types as T, f`
`rom pyspark.sql.window import Window as W.` `c` 는 컬럼 이름
또는 Column, `df` 는 DataFrame. **(A)** 는 액션(즉시 실행),
나머지는 변환(게으름)입니다.

B.1 세션·설정

코드	설명
<code>SparkSession.builder.appName(n).master(m).config(k, v).getOrCreate()</code>	세션 생성/재사용
<code>SparkSession.builder.remote("sc://host:15002")</code>	Spark Connect 클라이언트
<code>spark.conf.set(k, v) / spark.conf.get(k)</code>	런타임 설정
<code>spark.version</code>	Spark 버전
<code>spark.sparkContext.setLogLevel("WARN")</code>	로그 레벨
<code>spark.range(start, end, step, numPartitions)</code>	정수 id 컬럼 DataFrame
<code>spark.createDataFrame(data, schema)</code>	list/tuple/Row/dict/pandas → DataFrame
<code>spark.stop()</code>	세션 종료

B.2 읽기

코드	설명
<code>spark.read.format(f).option(k, v).schema(s).load(p)</code>	일반형
<code>spark.read.parquet(p) / .orc(p)</code>	컬럼 포맷
<code>spark.read.csv(p, header=True, schema=s)</code>	CSV. <code>sep</code> , <code>quote</code> , <code>escape</code> , <code>multiLine</code> , <code>nullValue</code>
<code>spark.read.json(p, multiLine=False)</code>	JSON Lines 기본
<code>spark.read.text(p)</code>	한 줄 = 한 행 (value)
<code>.option("mode", "PERMISSIVE" / "DROPMALFORMED" / "FAILFAST")</code>	불량 행 정책
<code>.option("recursiveFileLookup", True)</code>	하위 폴더까지
<code>.option("pathGlobFilter", "*.json")</code>	파일명 필터
<code>spark.read.table(t) / spark.table(t)</code>	카탈로그 테이블
<code>spark.read.format("jdbc").option("url", u).option("dbtable", t)</code>	JDBC. <code>partitionColumn</code> , <code>lowerBound</code> , <code>upperBound</code> , <code>numPartitions</code>
<code>spark.readStream...</code>	스트림 읽기

B.3 쓰기

코드	설명
<code>df.write.format(f).mode(m).option(k, v).save(p)</code>	일반형 (A)
<code>mode("overwrite" / "append" / "ignore" / "errorifexists")</code>	저장 모드
<code>df.write.parquet(p, compression="zstd")</code>	Parquet

코드	설명
<code>df.write.partitionBy("dt").parquet(p)</code>	Hive 스타일 파티션 디렉터리
<code>df.write.bucketBy(n, c).sortBy(c).saveAsTable(t)</code>	버킷 테이블
<code>df.write.saveAsTable(t)</code>	관리형 테이블
<code>df.write.insertInto(t, overwrite=False)</code>	기존 테이블에 위치 순서 로 삽입
<code>.option("maxRecordsPerFile", n)</code>	파일당 최대 행
<code>partitionOverwriteMode = dynamic</code>	대상 파티션만 덮어쓰기
<code>df.writeTo(t).append() / .overwritePartitions()</code>	DataSource V2 쓰기 API

B.4 살펴보기

코드	설명
<code>df.show(n=20, truncate=True, vertical=False)</code>	출력 (A)
<code>df.printSchema() / df.schema / df.dtypes / df.columns</code>	스키마
<code>df.count()</code>	행 수 (A)
<code>df.first() / df.head(n) / df.take(n) / df.tail(n)</code>	Row(들) (A)
<code>df.collect()</code>	전부 Driver로 (A) ⚠
<code>df.toPandas()</code>	pandas로 (A) ⚠
<code>df.toLocalIterator()</code>	파티션씩 순회 (A)
<code>df.limit(n)</code>	n행 DataFrame
<code>df.describe(*cols) / df.summary()</code>	요약 통계

코드	설명
<code>df.isEmpty()</code>	비었는지 (A)
<code>df.explain(mode="formatted")</code>	실행 계획
<code>df.isStreaming</code>	스트리밍 여부

B.5 컬럼 선택·추가·변경

코드	설명
<code>df.select(*cols) / df.select("...", expr)</code>	투영
<code>df.selectExpr("a", "b * 2 AS c")</code>	SQL 식으로 투영
<code>df.withColumn(name, expr)</code>	추가/교체
<code>df.withColumns({n1: e1, n2: e2})</code>	여러 개 한 번에
<code>df.withColumnRenamed(old, new)</code>	이름 변경
<code>df.withColumnsRenamed({old: new})</code>	여러 개 이름 변경
<code>df.toDF(*names)</code>	전체 이름 재지정
<code>df.drop(*cols)</code>	컬럼 삭제
<code>df.colRegex("^amt_.*")</code>	정규식으로 컬럼 선택
<code>df.alias("a")</code>	조인용 별칭
<code>df.transform(func, *args)</code>	사용자 함수로 체이닝
<code>df.unpivot(ids, values, var, val) / df.melt(...)</code>	wide → long
<code>df.withMetadata(c, dict)</code>	컬럼 메타데이터

B.6 행 필터·정렬·중복·샘플

코드	설명
<code>df.filter(cond) / df.where(cond)</code>	필터. SQL 문자열도 가능
<code>df.orderBy(*cols) / df.sort(...)</code>	전역 정렬 (셔플)
<code>F.asc(c), F.desc(c), F.desc_nulls_last(c)</code>	정렬 방향
<code>df.sortWithinPartitions(*cols)</code>	파티션 내 정렬 (셔플 없음)
<code>df.distinct()</code>	전체 중복 제거
<code>df.dropDuplicates(subset)</code>	키 기준 중복 제거
<code>df.dropDuplicatesWithinWatermark(subset)</code>	스트리밍 중복 제거
<code>df.sample(fraction, seed=)</code>	비율 샘플
<code>df.sampleBy(c, fractions={v: f}, seed=)</code>	층화 샘플
<code>df.randomSplit([0.8, 0.2], seed=)</code>	분할
<code>df.offset(n)</code>	앞 n행 건너뛰기

B.7 Column 메서드

코드	설명
<code>F.col(c), F.lit(v), F.expr(sql)</code>	컬럼·상수·SQL 식
<code>c.alias(n) / c.name(n)</code>	이름
<code>c.cast(t) / c.try_cast(t)</code>	형 변환 (실패 시 에러 / null)
<code>& ~, == != > >= < <=</code>	논리·비교 (괄호 필수)
<code>c.eqNullSafe(v)</code>	null 안전 비교 (<code><=></code>)
<code>c.isNull() / c.isNotNull() / c.isNaN()</code>	null·NaN 검사

코드	설명
<code>c.isin(*vals) / c.between(lo, hi)</code>	포함·범위
<code>c.like(p) / c.ilike(p) / c.rlike(re)</code>	패턴
<code>c.contains(s) / c.startswith(s) / c.endswith(s)</code>	부분 문자열
<code>c.substr(pos, len)</code>	부분 문자열 (1부터)
<code>c.getItem(k) / c[k] / c.getField(f)</code>	배열·맵·구조체 접근
<code>c.withField(f, e) / c.dropFields(f)</code>	구조체 필드 수정
<code>c.asc() / c.desc_nulls_last()</code>	정렬 방향
<code>c.over(window)</code>	윈도 적용
<code>c.when(...).otherwise(...)</code>	F.when 체이닝

B.8 조건·null

코드	설명
<code>F.when(cond, v).when(...).otherwise(v)</code>	CASE WHEN
<code>F.coalesce(*cols)</code>	첫 non-null
<code>F.ifnull(a, b) / F.nvl(a, b) / F.nvl2(a, b, c)</code>	null 대체
<code>F.nullif(a, b)</code>	a == b면 null
<code>F.nanvl(a, b)</code>	NaN 대체
<code>F.isnull(c) / F.isnotnull(c)</code>	함수형 null 검사
<code>F.greatest(*cols) / F.least(*cols)</code>	행 안 최대/최소 (null 무시)
<code>df.na.fill(v or dict, subset) / df.fillna(...)</code>	null 채우기

코드	설명
<code>df.na.drop(how="any", thresh=, subset=) / df.dropna(...)</code>	null 행 제거
<code>df.na.replace(old, new, subset) / df.replace(...)</code>	값 치환

B.9 수학·통계

코드	설명
<code>F.abs, F.round(c, d), F.bround, F.ceil, F.floor</code>	반올림 계열
<code>F.sqrt, F.pow(a, b), F.exp, F.log(c), F.log10, F.log1p</code>	거듭제곱·로그
<code>F.pmod(a, n) / a % n</code>	나머지
<code>F.try_add, F.try_subtract, F.try_multiply, F.try_divide</code>	실패 시 null (ANSI)
<code>F.rand(seed), F.randn(seed)</code>	난수
<code>F.monotonically_increasing_id()</code>	고유 ID (연속 아님)
<code>F.hash(*cols), F.xxhash64(*cols), F.md5, F.sha2(c, 256)</code>	해시
<code>F.format_number(c, d)</code>	천 단위 구분 문자열
<code>F.width_bucket(v, lo, hi, n)</code>	구간 번호
<code>df.stat.corr(a, b) / df.stat.cov(a, b)</code>	상관·공분산 (A)
<code>df.approxQuantile(c, [0.5], relErr)</code>	근사 분위수 (A)
<code>df.stat.crosstab(a, b) / df.stat.freqItems(cols)</code>	교차표·빈발값

B.10 문자열

코드	설명
<code>F.length</code> , <code>F.upper</code> , <code>F.lower</code> , <code>F.initcap</code>	기본
<code>F.trim</code> , <code>F.ltrim</code> , <code>F.rtrim</code> , <code>F.btrim(c, chars)</code>	공백 제거
<code>F.lpad(c, n, p)</code> , <code>F.rpad(c, n, p)</code>	채우기
<code>F.substring(c, pos, len)</code> , <code>F.left(c, n)</code> , <code>F.right(c, n)</code>	부분 문자열
<code>F.substring_index(c, delim, n)</code>	구분자 n번째까지
<code>F.concat(*cols)</code> , <code>F.concat_ws(sep, *cols)</code>	연결
<code>F.split(c, regex, limit)</code>	배열로 분리
<code>F.split_part(c, delim, n)</code>	n번째 조각
<code>F.regexp_extract(c, re, idx)</code> / <code>F.regexp_extract_all</code>	정규식 추출
<code>F.regexp_replace(c, re, rep)</code>	정규식 치환
<code>F.regexp_like(c, re)</code> / <code>F.regexp_count</code>	정규식 판정·횟수
<code>F.replace(c, s, r)</code> , <code>F.translate(c, from, to)</code>	문자 치환
<code>F.instr(c, s)</code> , <code>F.locate(s, c)</code> , <code>F.contains(c, s)</code>	위치·포함
<code>F.format_string(fmt, *cols)</code> , <code>F.printf</code>	printf 서식
<code>F.repeat(c, n)</code> , <code>F.reverse(c)</code>	반복·뒤집기
<code>F.base64</code> , <code>F.unbase64</code> , <code>F.encode</code> , <code>F.decode</code>	인코딩
<code>F.levenshtein(a, b)</code> , <code>F.soundex(c)</code>	유사도

코드	설명
<code>F.to_number(c, fmt), F.to_char(c, fmt)</code>	숫자 ↔ 서식 문자열
<code>F.parse_url(url, "HOST"), F.url_decode(c)</code>	URL

B.11 날짜·시간

코드	설명
<code>F.current_date(), F.current_timestamp(), F.now()</code>	현재
<code>F.to_date(c, fmt), F.to_timestamp(c, fmt)</code>	파싱 (yyyy-MM-dd HH:mm:ss)
<code>F.try_to_timestamp(c, fmt)</code>	파싱 실패 시 null
<code>F.date_format(c, fmt)</code>	서식 문자열로
<code>F.year, F.quarter, F.month, F.weekofyear, F.dayofmonth</code>	필드 추출
<code>F.dayofweek (1=일), F.weekday (0=월), F.dayofyear</code>	요일
<code>F.hour, F.minute, F.second, F.extract("DOW", c)</code>	시간 필드
<code>F.date_add(c, n), F.date_sub(c, n), F.add_months(c, n)</code>	날짜 연산
<code>F.datediff(end, start), F.months_between(a, b)</code>	차이
<code>F.timestamp_add("HOUR", n, c), F.timestamp_diff("MINUTE", a, b)</code>	단위 지정 연산
<code>F.trunc(c, "month"), F.date_trunc("hour", c)</code>	절단
<code>F.last_day(c), F.next_day(c, "Mon")</code>	월말·다음 요일

코드	설명
<code>F.make_date(y, m, d), F.make_timestamp(...)</code>	조립
<code>F.unix_timestamp(c), F.from_unixtime(n, fmt)</code>	Unix 초
<code>F.timestamp_seconds(n), F.unix_millis(c)</code>	초·밀리초 변환
<code>F.from_utc_timestamp(c, tz), F.to_utc_timestamp(c, tz)</code>	타임존 변환
<code>F.window(ts, "10 minutes", slide)</code>	시간 버킷
<code>F.session_window(ts, "30 minutes")</code>	세션 버킷
<code>F.sequence(start, stop, F.expr("INTERVAL 1 DAY"))</code>	날짜 시퀀스 배열

B.12 배열·맵·구조체·JSON

코드	설명
<code>F.array(*cols), F.create_map(k1, v1, ...), F.struct(*cols)</code>	생성
<code>F.size(c), F.array_contains(c, v), F.array_position(c, v)</code>	크기·포함·위치
<code>F.get(c, i) (0부터), F.element_at(c, i) (1부터), F.try_element_at</code>	원소 접근
<code>F.explode, F.explode_outer, F.posexplode, F.inline</code>	배열 → 행
<code>F.array_distinct, F.array_union, F.array_intersect, F.array_except</code>	집합 연산
<code>F.array_sort, F.sort_array(c, asc), F.array_max, F.array_min</code>	정렬·최대최소

코드	설명
<code>F.array_join(c, sep), F.flatten(c), F.slice(c, start, len)</code>	결합·평탄화·자르기
<code>F.array_append, F.array_prepend, F.array_insert, F.array_remove, F.array_compact</code>	원소 추가·제거
<code>F.arrays_zip(*cols), F.sequence(a, b), F.shuffle(c)</code>	묶기·수열·섞기
<code>F.transform(c, f), F.filter(c, f), F.aggregate(c, init, f)</code>	고차 함수
<code>F.exists(c, f), F.forall(c, f), F.zip_with(a, b, f)</code>	고차 함수
<code>F.map_keys, F.map_values, F.map_entries, F.map_from_entries</code>	맵 분해
<code>F.map_from_arrays(k, v), F.map_concat, F.map_filter, F.transform_values</code>	맵 조작
<code>F.from_json(c, schema), F.to_json(c), F.schema_of_json(s)</code>	JSON ↔ 구조체
<code>F.get_json_object(c, "\$.a.b"), F.json_tuple(c, *fields)</code>	JSON 필드 추출
<code>F.parse_json(c), F.variant_get(v, path, type), F.try_variant_get</code>	VARIANT (4.0+)
<code>df.select("s.*")</code>	구조체 펼치기

B.13 집계

코드	설명
<code>df.groupBy(*cols).agg(*exprs)</code>	그룹 집계
<code>df.groupBy(...).count() / .sum(c) / .avg(c) / .max(c)</code>	단축형

코드	설명
<code>df.agg(*exprs)</code>	전체 집계
<code>df.groupBy(k).pivot(c, values).agg(...)</code>	피벗
<code>df.rollup(*cols) / df.cube(*cols) + F.grouping(c)</code>	소계
<code>df.groupingSets(..., *cols)</code>	임의 그룹 조합 (4.0+)
<code>F.count("*"), F.count(c), F.count_if(cond)</code>	개수
<code>F.sum, F.avg/F.mean, F.min, F.max, F.product</code>	기본
<code>F.countDistinct(*c), F.approx_count_distinct(c, rsd)</code>	고유값 수
<code>F.sum_distinct(c)</code>	고유값 합
<code>F.collect_list(c), F.collect_set(c), F.array_agg(c)</code>	배열로 모으기
<code>F.first(c, ignorenulls), F.last(c, ...), F.any_value(c)</code>	대표값 (순서 무보장)
<code>F.max_by(x, y), F.min_by(x, y)</code>	y 최대/최소 행의 x
<code>F.median(c), F.mode(c), F.percentile_approx(c, p), F.percentile(c, p)</code>	분위수
<code>F.stddev, F.variance, F.skewness, F.kurtosis, F.corr(a, b)</code>	통계
<code>F.bool_and(c), F.bool_or(c) / F.every, F.some</code>	논리 집계
<code>F.listagg(c, sep) / F.string_agg</code>	문자열 연결 집계 (4.0+)

B.14 윈도우

코드	설명
<code>W.partitionBy(*cols).orderBy(*cols)</code>	윈도우 정의
<code>.rowsBetween(start, end)</code>	행 개수 프레임
<code>.rangeBetween(start, end)</code>	값 범위 프레임
<code>W.unboundedPreceding, W.currentRow, W.unboundedFollowing</code>	프레임 경계
<code>F.row_number(), F.rank(), F.dense_rank()</code>	순위
<code>F.percent_rank(), F.cume_dist(), F.ntile(n)</code>	분위 순위
<code>F.lag(c, n, default), F.lead(c, n, default)</code>	앞뒤 행
<code>F.nth_value(c, n, ignoreNulls)</code>	n번째 값
<code>F.first(c, True).over(w), F.last(c, True).over(w)</code>	프레임 첫/끝 (ffill 용도)
<code>F.sum(c).over(w)</code> 등 모든 집계 함수	윈도우 집계

B.15 조인·집합 연산

코드	설명
<code>df.join(o, on, how)</code>	<code>on</code> : 이름/리스트/Column 식
<code>how="inner" / "left" / "right" / "full"</code>	기본 조인
<code>how="left_semi" / "left_anti"</code>	존재/부재 필터
<code>df.crossJoin(o)</code>	카티션 곱
<code>F.broadcast(o) / o.hint("broadcast")</code>	브로드캐스트 힌트

코드	설명
<code>o.hint("merge" / "shuffle_hash")</code>	조인 전략 힌트
<code>df.lateralJoin(o, on, how)</code>	래터럴 조인 (4.0+)
<code>df.unionByName(o, allowMissingColumns=True)</code>	이름 기준 합치기
<code>df.union(o)</code>	위치 기준 합치기 ⚠
<code>df.intersect(o) / df.intersectAll(o)</code>	교집합
<code>df.subtract(o) / df.exceptAll(o)</code>	차집합

B.16 파티션·캐시·실행

코드	설명
<code>df.rdd.getNumPartitions()</code>	파티션 수
<code>df.repartition(n, *cols)</code>	재분배 (셔플)
<code>df.repartitionByRange(n, *cols)</code>	범위 분할 (셔플)
<code>df.coalesce(n)</code>	파티션 병합 (셔플 없음)
<code>df.cache() / df.persist(StorageLevel.X)</code>	캐시 표시
<code>df.unpersist() / spark.catalog.clearCache()</code>	캐시 해제
<code>df.storageLevel</code>	캐시 상태
<code>df.checkpoint() / df.localCheckpoint()</code>	계보 끊기 (A)
<code>df.explain() / explain(mode="extended"/"formatted"/"cost")</code>	계획 보기
<code>df.hint(name, *params)</code>	옵티마이저 힌트
<code>F.spark_partition_id()</code>	행이 속한 파티션 번호

코드	설명
<code>df.foreach(f) / df.foreachPartition(f)</code>	부수효과 실행 (A)

B.17 SQL·카탈로그

코드	설명
<code>df.createOrReplaceTempView(n)</code>	세션 임시 뷰
<code>df.createOrReplaceGlobalTempView(n)</code>	<code>global_temp.n</code>
<code>spark.sql(q)</code>	SQL 실행 → DataFrame
<code>spark.sql(q, args={"p": v}) / :p</code>	파라미터 바인딩
<code>spark.sql("... {t} ...", t=df)</code>	DataFrame 치환
<code>spark.catalog.listDatabases() / listTables(db) / listColumns(t)</code>	조회
<code>spark.catalog.tableExists(t)</code>	존재 확인
<code>spark.catalog.dropTempView(n)</code>	뷰 삭제
<code>spark.catalog.refreshTable(t)</code>	메타데이터 새로고침
<code>spark.catalog.setCurrentDatabase(db)</code>	기본 DB
<code>ANALYZE TABLE t COMPUTE STATISTICS</code>	통계 수집 (CBO)
<code>FROM t \ > WHERE ... \ > AGGREGATE ...</code>	SQL 파이프 문법 (4.0+)

B.18 UDF·pandas

코드	설명
<code>@F.udf(returnType)</code>	Python UDF
<code>@F.udf(returnType, useArrow=True)</code>	Arrow 직렬화 UDF

코드	설명
@F.pandas_udf(returnType) + Series -> Series	벡터 UDF
@F.pandas_udf(t) + Iterator[Series] -> Iterator[Series]	반복자 UDF (초기화 1회)
@F.pandas_udf(t) + Series -> scalar	집계 UDF
spark.udf.register(name, f, t)	SQL에서 쓸 이름 등록
@F.udtf(returnType=...)	테이블 함수 (행 → 여러 행)
df.groupBy(k).applyInPandas(f, schema)	그룹별 pandas
df.mapInPandas(f, schema)	파티션별 pandas (셔플 없음)
a.groupBy(k).cogroup(b.groupBy(k)).a.applyInPandas(f, s)	두 DF 그룹 공동 처리
df.mapInArrow(f, schema) / applyInArrow	Arrow 배치 직접 처리
df.pandas_api() / psdf.to_spark()	pandas API on Spark 변환
spark.conf.set("spark.sql.execution.arrow.pyspark.enabled", True)	Arrow 변환

B.19 스트리밍

코드	설명
spark.readStream.format(f).schema(s).option(...).load(p)	소스 (kafka , json , parquet , rate ...)
df.withWatermark(ts, "10 minutes")	늦은 데이터 한도
df.writeStream.format(f).outputMode(m).option("checkpointLocation", p).start(p)	시작 → StreamingQuery

코드	설명
<code>outputMode("append" / "update" / "complete")</code>	출력 모드
<code>.trigger(processingTime="1 minute") / .trigger(availableNow=True)</code>	트리거
<code>.foreachBatch(func)</code>	배치마다 임의 처리
<code>.toTable(t)</code>	테이블로 스트리밍 쓰기
<code>q.awaitTermination() / q.stop()</code>	대기·중지
<code>q.status / q.lastProgress / q.recentProgress</code>	상태·진행
<code>spark.streams.active</code>	실행 중 쿼리

B.20 테스트·기타

코드	설명
<code>from pyspark.testing import assertDataFrameEqual</code>	DF 비교 (순서 무시 기본)
<code>assertSchemaEqual(a.schema, b.schema)</code>	스키마 비교
<code>T.StructType.fromDDL("a INT, b STRING")</code>	DDL → 스키마 객체
<code>df.schema.simpleString() / df.schema.json()</code>	스키마 직렬화
<code>from pyspark.errors import AnalysisException, PySparkException</code>	예외 클래스
<code>e.getCondition() / e.getMessageParameters()</code>	에러 분류·파라미터
<code>spark.sparkContext.broadcast(v) / .accumulator(0)</code>	브로드캐스트 변수·어큐뮬레이터

코드	설명
<code>spark.sparkContext.addPyFile(p)</code>	Executor에 Python 파일 배포